

Multi-Scenario Cloud–Edge Collaborative DDoS Detection in LLM-Enabled AIoT

Aoran Huang[✉], Graduate Student Member, IEEE, Jingfu Yan[✉], Xiaojing Fan[✉], and Huachun Zhou[✉]

Abstract—DDoS attack detection has become crucial for securing the Internet of Things (IoT). However, next-generation intelligent networks produce large-scale, heterogeneous data, and intrusion-detection systems built on conventional AI models lack the capacity to provide targeted detection for diverse IoT application scenarios. Large language models (LLMs) have attracted attention due to their strong ability to capture contextual relationships. Given the correlated nature of DDoS flows, using LLMs for scenario-specific DDoS detection across heterogeneous IoT applications is a promising direction. Nevertheless, two main challenges remain: (1) how to enable LLMs to process non-textual DDoS flow data and to leverage edge models' detection capabilities within AI-IoT (AIoT) scenarios; and (2) how to design a detection framework that harnesses LLM strengths in large-scale, multi-type data handling to accurately identify diverse attack types across IoT application scenarios. To this end, we propose CE4LDD, a cloud–edge collaborative method for detecting DDoS attacks based on an LLM architecture. We design an order-assignment module, a token generation group (TGG), and an embedding module for flow data. DDoS detection is formulated as a token-classification task within the LLM. During tokenization, we leverage edge models' detection capabilities. Moreover, to support diverse AIoT scenarios, we present EAL-BT, a bottom-up training and detection process. We describe how edge models assist the LLM in scenario-specific detection and provide CE4LDD's complete training and detection workflow. Finally, using CIC2019 and a custom dataset, we simulate five IoT scenarios under different DDoS attacks. Experimental results show that the proposed method can help LLM perform DDoS detection in AIoT and achieve excellent performance in handling DDoS attacks across different IoT application scenarios.

Index Terms—Large Language Model, AIoT Security, DDoS Intelligent Intrusion Detection, Multi-Scenario, Intelligent Network Security.

I. INTRODUCTION

BECAUSE edge devices in IoT deployments are directly exposed to the Internet, they are more susceptible to network attacks [1]. With the advent of next-generation intelligent network technology [2], [3], the access of a large number of widely distributed devices with certain bandwidth resources [4] not only enriches the application scenarios of the IoT, but also

provides a hotbed for DDoS attacks [5]. According to research from Nexusguard and Nokia, DDoS attacks targeting IoT surged by 233% year-on-year from 2023 to 2024 [6], with daily attack frequency increasing from once per day to over 100 times [7], underscoring that DDoS has become the primary means of compromising IoT systems [8]. Attackers launch different types of DDoS attacks according to the characteristics of different IoT application scenarios [9]. For example, in the consumer IoT scenario [10] with the main purpose of hijacking and stealing, botnets are often used for attacks [11]; in the vehicular network [12], DrDoS is a commonly used attack type [13]. Considering the wide application scenarios of the IoT, whether it is the loss of important flow data caused by the edge device being controlled as a zombie machine [14], or the server-side device being attacked and affecting the normal operation of the entire network [15], the risks and costs are unacceptable. Therefore, targeted detection of DDoS attacks in different IoT scenarios is the key to ensuring the security of the IoT [16].

In the face of large-scale flow data caused by the complexity of the IoT architecture and scale [17], how to design a detection scheme to detect different DDoS attacks in different application scenarios [18] has become a key research issue [19], [20]. The application scenarios of different edges in a cloud may be different. Existing solutions primarily deploy AI models at the edge, enabling different DDoS detection across multiple scenarios via techniques like knowledge sharing [21] and federated learning (FL) [22]. Or deploy the model directly on the cloud/edge side and train the model with a large amount of hybrid DDoS data [23]. These methods incur heavy communication and computation costs during training, and often suffer from overfitting, slow convergence, and poor generalization.

The emergence of Large Language Model (LLM) provides a new way to solve the above problems [24], [25]. Its powerful performance has been proved in natural language processing [26], multimodal [27], [28] and other aspects, and has begun to explore its application in wireless communication [29], [30], IoT [31], IoV [32] and other fields. Especially in IoT, numerous recent studies on the agent mode of LLM in AI-IoT (AIoT) [33], distributed reinforcement learning methods [34], and industrial IoT (IIoT) applications [35] have demonstrated the adaptability of LLM in AIoT networks [36]. Leveraging vast training data and its attention mechanism [37], LLM outperforms conventional AI models by capturing broader relationships and characteristics among data, making it well-suited for handling large-scale, diverse flow data across various AIoT scenarios. Combined with the characteristics of DDoS attacks that there is a certain correlation between their flow, such as frequent access to the same address, multiple use of the same protocol, etc., it is reasonable to use LLM to detect DDoS attacks in AIoT.

Received 28 July 2025; revised 13 October 2025; accepted 19 November 2025. Date of publication 27 November 2025; date of current version 24 December 2025. This work was supported in part by the National Science Foundation of China under Grant U2569201 and Grant 62341102 and in part by the National Key R&D Program of China under Grant 2018YFA0701604. Recommended for acceptance by Dr. Liehuang Zhu. (Corresponding author: Huachun Zhou.)

The authors are with the School of Electronic and Information Engineering, Beijing Jiaotong University, Beijing 100044, China (e-mail: huangaoran@bjtu.edu.cn; yanjingfu@bjtu.edu.cn; xiaojing_fan@bjtu.edu.cn; hchzhou@bjtu.edu.cn).

Digital Object Identifier 10.1109/TNSE.2025.3637740

However, considering the application scenarios of LLM, the resource distribution of the IoT and the needs of actual DDoS detection, this paper reviews a large number of recent studies on DDoS detection in AIoT and the application of LLM in IoT, mainly facing the following two difficulties:

- Considering the resource constraints at the AIoT edge [38], LLM can only be deployed in the cloud, while edge models retain valuable capabilities for detecting prevalent DDoS attacks. How can these edge models be leveraged to assist LLM in DDoS detection? Additionally, since LLMs are designed primarily for textual data—with tokenization, embedding, and associated processes tailored to natural language—how can non-textual, unordered flow data be selected and preprocessed for ingestion by LLM, thus utilizing its robust capabilities for flow classification [39].
- Edges under the same cloud may serve different application scenarios [40], and consequently face different types of DDoS attacks [41]. For example, smart wearable scenarios and car networking scenarios can be connected to the same cloud through different edge terminals. However, the main DDoS attacks on the two are different. How to design a detection scheme based on the advantages of LLM in processing large-scale and multi-type data, so that it can make targeted attack detection for different types of attacks in different AIoT scenarios.

To address these challenges, this paper first proposes a cloud-edge collaboration method based on LLM to detect DDoS attack flow, named CE4LDD (Cloud Edge for LLM DDoS Detection). It describes in detail how cloud-edge collaboration processes flow data according to LLM characteristics, enabling flow classification using the LLM's capabilities. Its edge function is provided by the conventional AI model, and the cloud function is composed of five parts: order generation module, token selection gating (T-Gate), token generation group (TGG), embedding module and LLM without mask. Considering the correlation between DDoS attack flow [42], the module is designed to give the flow order [43] and generate the flow order matrix. According to the different types of attacks, the appropriate TGG is selected by T-Gate based on weight entropy. The TGG is constructed from aggregated edge models and leverages their detection capabilities. Compared with the tokenization of text data, there is no padding tokens added in the process of flow data tokenization, which is more suitable for LLM to process large-scale and diversified flow data. The embedding module maps generated flow tokens into a high-dimensional space for input to the LLM, enabling analysis of richer high-dimensional features. We disable the LLM's masking mechanism so it can attend to all input flow tokens; DDoS detection is formulated as a token-classification task within the LLM, and final predictions are produced by a linear projection layer.

To tailor CE4LDD for AIoT environments and enable scenario-specific DDoS detection, a bottom-up [44] edge assisted LLM training and detection process called EAL-BT (Edge Assist LLM-Bottom to Top) is proposed. The architecture can use the targeted detection ability of the edge model to help the detection of LLM. For the edge, edge clusters are defined according to the division of application scenarios. The edge model in each cluster is trained based on the flow data of the subordinate IoT devices, so that it has a certain DDoS detection capability. Models in the same cluster are aggregated into the cloud to construct a TGG, and the last hidden-layer outputs of these edge models are combined to generate tokens corresponding to different DDoS

attack types, so as to realize the help of the edge model detection ability to the LLM in the cloud to detect DDoS attack flow. In addition, the architecture also stipulates the training sequence of multiple models in the cloud and edge of the detection system and the transmission rules of flow in the detection process. This paper gives the complete training and testing process, model and data transmission direction of EAL-BT architecture. The main contributions of this paper are as follows:

- A large number of studies on DDoS detection in AIoT and the application of LLM in IoT are reviewed. The key information such as the technical scheme, scenario-oriented, DDoS attack type, and data set used are summarized. The limitations of the current research and the differences and advantages of the proposed scheme are analyzed.
- A cloud-edge collaboration method named CE4LDD based on LLM to detect DDoS attack flow is proposed. To enable the LLM to handle non-text DDoS flow data, we order the flows and design tokenization and embedding methods that allow the LLM to bypass masking and analyze correlations in large-scale flow data. The DDoS detection task is corresponding to the token classification task to achieve flow token classification. In order not to waste the detection ability in the edge, the edge model is used to construct a token generation group to assist the LLM in the cloud to analyze the characteristics of the flow.
- We propose EAL-BT, a bottom-up training and detection process for AIoT that enables CE4LDD to perform targeted DDoS detection according to varying attack preferences across application scenarios. The complete process is given. In the training process, the training order: edge model $\rightarrow$ embedding module, linear projection layer $\rightarrow$ T-Gate is specified, and the loss function of embedding model, linear projection layer and T-Gate training is given. In the detection process, the transmission rules of flow data in the detection process are given based on the trusted entropy. The supervision and detection part is designed to ensure that the detection system can be updated in time.
- CIC2019 and self-dataset are used to simulate five IoT scenarios under different DDoS attacks, including DrDoS, LDDoS, AppDDoS, NetDDoS and Botnet. The comparative experiments with the other five models show that the proposed scheme has obvious advantages in terms of accuracy, recall and F1-score in the face of unbalanced attacks and hybrid attacks. The ablation experiment proves that the proposed order assignment module can help LLM perform DDoS detection better, and the bottom-up training process can accelerate the convergence speed.

The remainder of this paper is composed as follows. In Section II, the related work and preliminaries are firstly introduced. In Section III, the architecture, function and implementation of CE4LDD are introduced. In Section IV, the complete processes of EAL-BT is introduced. The Section V describes the simulation experiment. Finally, the Section VI provides a summary of this paper.

II. RELATED WORK AND PRELIMINARIES

This section introduces the DDoS detection in AIoT and the application of LLM in AIoT. In preliminaries, the basic idea of using LLM to process flow data, as well as DDoS classification and its correlation, are introduced.

A. Related Work

1) *DDoS Detection in AIoT*: The development of next generation network technology provides support for ubiquitous collaboration and connectivity [45], and will also bring more opportunities for DDoS attacks [46]. The rise of complex application scenarios, such as satellite-terrestrial integrated networks [47], has compelled researchers to account for scenario-specific characteristics. For IoT, different IoT application scenarios and types are vulnerable to different types of DDoS attacks. Firstly, some work on DDoS attacks in a single IoT scenario or type is sorted out. X. He et al. [48] et al. simulated the DNS attack of DrDoS in the ship IoT scenario based on Markov chain, proved that the IoT in the marine network is vulnerable to DNS rebinding attacks, and gave the detection and defense system by controlling access rights. Z. Liu et al. [49] analyzed the characteristics of LDDoS in SDN-IoT, and designed a double sliding method with expansion filling to deal with LDDoS flow. Combined with CNN and LSTM, a detection method of LDDoS is proposed. Yin.W et al. [50] proposed a feature optimization method for LDDoS detection by combining clustering algorithm and decision number to filter traffic features for SDN-IoT network features. For the application scenario of smart grid, M.N.Kurt et al. [51] took UDP Flood attack and spam attack in NetDDoS as the detection objects in this scenario, and proposed an online privacy-preserving anomaly detection method through data-driven and differential privacy methods. Z. Tian et al. [52] analyzed AppDDoS attacks in Web of things and used deep learning to detect AppDDoS attacks based on various URL representation methods. Jia. Y [53] used GNN to analyze the characteristics of DDoS attacks initiated by various applications and their characteristics for smart home scenarios, and proposed an improved RCNN user activity inference to help AppDDoS detection. X. Yan et al. [54] analyzed the enhancement effect of smart contracts on botnets in the smart blockchain scenario, and proposed a botnet detection method based on domain embedding and unsupervised learning. W. Ma et al. [55] proposed a lightweight botnet detection method based on the proposed cloud-edge-node framework in resource-constrained IoT scenarios. A. Sharma et al. [56] discussed a fundamental component of IDS based on the important parameter security in intelligent healthcare scenarios by combining deep learning, CNN and bidirectional LSTM. An intrusion detection method for multiple attacks including botnets is designed. A. Ali et al. [57] designed a homomorphic encryption model based on hybrid deep learning and combined with blockchain to achieve privacy protection and network attack resistance in the industrial medical Internet of Things scenario. M. F. Saiyedand et al. [58] integrated CNN and LSTM to detect hybrid DDoS attacks, and optimized the integrated model by pruning to facilitate deployment on resource-constrained IoT devices.

Beyond DDoS analysis and detection in single IoT scenarios, distributed architectures or cloud-edge collaboration have been proposed to address detection needs across diverse IoT application scenarios [59]. H. Feng et al. [60] designed a multi-agent reinforcement learning method supporting GNN in SDN-IoT for path selection to mitigate DDoS attacks. This paper theoretically supports multi-scenario DDoS detection, and agent supports LLM access, but there is no further design and discussion. The three schemes of MFGD3QN [61], FOSS [62] and FIEAM [63] meet the needs of DDoS detection in different application scenarios of IoT through distributed architecture,

segmentation learning and FL respectively. MFGD3QN [52] proposed a multi-agent deep reinforcement learning algorithm (MADRL) by constructing the mean field game model (MFG), and realized the DDoS defense strategy in the edge intelligent scene. FOSS [62] used the isolation tree as the backbone to construct a model to segment the data distribution hyperplane to detect unknown DDoS attacks in IoT. However, neither of them can take advantage of the detection ability of the edge model. FIEAM [63] uses distributed data sets to train the global model through FL and edge computing to mitigate DDoS attacks, and realizes the use of edge models while supporting multi-scene detection. However, since the model that needs to be transmitted to the edge in the cloud is too large, the scheme cannot support LLM. R. F. Hayat et al. [64] proposed a multi-level DDoS mitigation method that uses blockchain verification mechanisms to blocklist suspicious devices to prevent them from being controlled. In addition, compared with the proposed methods, although the above method support multi-scenario DDoS attack detection, they essentially still use hybrid DDoS attacks to train a common model to sink to the edge. The edge does not have the ability to detect a certain DDoS attack, and the ability of the edge model is not fully utilized. Our scheme can detect various types of DDoS attacks according to different scenarios.

Table I provides an overview of the above methods, including model methods, IoT scenarios/types, types of DDoS attacks that can be detected, edge model utilization capabilities, use of data sets and whether LLM is supported. Among them, it is difficult to support LLM access due to incompatibility with LLM in some aspects.

2) *LLM for IoT*: Recent research on the combination of LLM and next-generation network communication has proved the feasibility of LLM enabling multi-type networks [65], and also provided a basis for deploying LLM for IoT [66], [67]. With the development of multi-agent technology [68], helping IoT processing tasks through Agent mode is a major way for LLM to be applied in IoT [69]. Rivkin et al. [31] triggered LLM control discrete program based on user request to realize agent function in smart home scenario. H. Cui et al. [70] proposed an AI-oriented proxy framework, which implements language-to-code conversion based on LLM to generate control scripts to orchestrate complex task execution in IoT. Facing the IoT of multiple application scenarios, Zhong et al. [71] designed the LLM agent system through the memory and summary module, and provided strategies for different application scenarios based on prompts and historical data. B. Xiao et al. [72] proposed a LLM-based generative IoT system that uses prompt enhanced LLM to help IoT handle tasks in different application scenarios. Based on generative adversarial networks and self-learning, Tu et al. [73] proposed the Seafarer architecture, which solves the problem of model deviation when using heterogeneous data to fine-tune LLM to help deal with unknown tasks in AIoT. N.-B.-V.Le et al. [74] added LLM to the auxiliary layer to help users analyze different environmental conditions and data. A. Selim et al. [75] used LLM to detect the curve manipulation of Volt/VAR based on text control command to realize network attack detection in smart grid. B. Karunanayake et al. [76] proposed an adaptive IDS policy orchestration framework without LLM reasoning in the policy execution phase by combining LLM Agent with advanced prompt policy in real-time IoT, which meets the delay-sensitive requirements of real-time IoT.

The above research mainly focuses on the agent application of LLM in IoT, while another active research area involves

TABLE I
COMPARISON OF DDOS DETECTION SCHEMES IN AIOT

Reference	Scenario/Type	Attack Type	Approach	Edge Utilization	Dataset	LLM Support
X. He et al. [48]	Maritime IoT	DrDoS	Access Control	×	Custom	×
Z. Liu et al. [49]	SDN-IoT	LDDoS	CNN+LSTM	×	ISCX-2016-SlowDos	×
Yin. W ta al. [50]	SDN-IoT	LDDoS	Clustering Algorithm + Structural Tree	×	Custom	×
M. N. Kurt et al. [51]	Smart Grid	NetDDoS	Data-driven + Differential Privacy	×	N-BaIoT	×
Z. Tian et al. [52]	Web of Things	AppDDoS	Deep Learning	×	CSIC 2010 + Fwaf+HttpParams	×
Jia. Y [53]	Smart Home	AppDDoS	GNN+RCNN	×	CICDDoS2019 + Csutom	×
X. Yan et al. [54]	Smart Blockchain	Botnet	Domain Embedding + Unsupervised Learning	×	Custom	×
W. Ma et al. [55]	Constrained IoT	Botnet	CNN	×	IEEE Dataport IoT Network Intrusion	×
A. Sharma et al. [56]	Smart Healthcare	Botnet	Deep Learning + CNN + LSTM	×	BoT-IoT	×
A. Ali et al. [57]	IIoMT	Botnet	Hybrid DL + Blockchain	Blockchain	Custom	×
M. F. Saiyedand et al. [58]	Constrained IoT	Hybrid DDoS	Ensemble Learning + Pruning	×	HL-IoT	×
H. Feng et al. [60]	SDN-IoT	Hybrid DDoS	GNN + RL	Possibly Supported	Custom	Possibly Supported
MFGD3QN [61]	Multi-scenario	Hybrid DDoS	MFG + MADRL	×	Custom	×
FOSS [62]	Multi-scenario	Hybrid DDoS	ML + Split Learning	×	IDS2017 + VPN2016	×
FLEAM [63]	Multi-scenario	Hybrid DDoS	GRU + FL	FL	Custom	×
R. F. Hayat et al. [64]	Multi-scenario	Hybrid DDoS	Control Access + Blockchain	Blockchain	Custom	×
Our	Multi-scenario	Adaptive	CE4LDD + EAL-BT	Bottom-up	CIC2019 + Custom	✓

leveraging LLM's architecture to address network issues in IoT via cloud-edge collaborative fine-tuning and inference [77]. M. Fu et al. [78] pre-trained and fine-tuned traffic data by assigning context semantics to traffic data, combined with targeted data sets. Using ability of LLM to process context, an LLM-based IDS is given in IoV, which can detect attacks in a dynamic environment. Q. Liu et al. [79] used LLM to learn multiple types of data, and generated a real-time RIS model by letting LLM learn traffic data such as channels and vehicles. Based on this model, a wireless resource allocation strategy is given. R. Zhang et al. [80] discussed the integration method of generative AI (GAI) in vehicle networks, and proposed a multi-modal semantic awareness framework to support multi-modal content generation to guide system operation. G. Sun et al. [81] proposed a new GAI framework to improve communication and security performance in UAV networking scenarios. The combination of LLM and federated learning is a typical example. Federal LLM is systematically introduced by J. Hu et al. [82], including its paradigm, workflow and future challenges. S. Han et al. [83] proposed an attack defense method named FedSecurity based on FL and gave a joint LLM training method. F. Wu et al. [84] refined public datasets through FL to adapt to private datasets, helping users with insufficient data to achieve privacy-preserving fine-tuning of LLM. C. Han et al. [85] proposed a hierarchical federated collaborative computing framework. By dynamically splitting the LLM into a global shared layer and a device native layer, the inference speed and malicious node detection performance in resource-constrained environments are improved. Aiming at special IoT application scenarios such as IIoT, considering its problems of resource coordination and delay [86], Y. Liu et al. [87] used LLM as a supervisor to realize fault diagnosis in IIoT through vertical federated learning (VFL) combined with brain-like thinking chain. Y. Xia et al. [88] assisted LLM to generate virtual data sets in the case of sensor failure

by retrieval enhancement technology, and used local models to enhance global models with federated continuous learning (FCL) to improve the accuracy of fault diagnosis in IIoT.

Table II summarizes the application of LLM model in different IoT scenarios. It is worth noting that although this paper also directly uses the LLM architecture to empower AIoT, the method proposed in this paper is different from FL. FL is to help each node learn global knowledge through a top-down architecture, so that the size of the model transmitted to the edge is large. As a result, LLM in the cloud needs to be pruned or transmitted some parameters to send to the edge, which makes it difficult to give full play to the performance of LLM. Our method is a bottom-up architecture that uses global nodes to provide knowledge help for large models in the cloud and avoids problems in FL. The existing scheme is mainly based on the language processing ability of LLM and deploys it as an expert model that can answer questions in AIoT for tasks such as text analysis and answering. The data format processed is limited to text data. The proposed scheme directly uses LLM to empower AIoT, and uses LLM as a large neural network model to process the input non-text format flow data.

B. Preliminaries

1) *LLM for Flow Data*: LLM consists of four parts: Tokenization, embedding, LLM backbone and LLM task. The tokenization part divides the input text, and adds padding token to cooperate with the mask mechanism in the LLM backbone to ensure that LLM cannot observe the subsequent input and ensure the reliability of the model. Embedding is responsible for vectorizing the generated token and mapping it to a high-dimensional space for feature analysis. The LLM skeleton part is based on multiple Transformers to capture the order and context correlation between tokens. Its essence is also composed

TABLE II
COMPARISON OF APPLICATION SCHEMES OF LLM IN AIoT

Reference	Scenario	Usage	Data Format	Function
D. Rivkin et al. [31]	Smart home	Agent	Text	User Interaction
H. Cui et al. [70]	AIoT	Agent	Text	Task Orchestration
N. Zhong et al. [71]	AIoT	Agent	Text	Multi-scenario Intelligence
B. Xiao et al. [72]	Generative IoT	Agent	Tabular Text	Q&A Task
T. Tu et al. [73]	AIoT	Agent	Text	Unknown Task
N.-B.-V. Le et al. [74]	Smart Agriculture	Agent	Text	Q&A Task
A. Selim et al. [75]	Smart Grid	Agent	Text	Cybersecurity
B. Karunanayake et al. [76]	Real-time IoT	Agent	FLow Data	IDS
M. Fu et al. [78]	IoV	Enabling	FLow Data	IDS
Q. Liu et al. [79]	IoV	Enabling	Traffic Environment Data	RIS Model Construction
R. Zhang et al. [80]	V2V	Enabling	Multi-Modal Data	Vehicle Guidance Content Generation
G. Sun et al. [81]	UAV	Enabling	Network Environment Data	Spectrum estimation content generation
[82] [83] [84]	FL	Enabling		
C. Han et al. [85]	Constrained IoT + FL	Enabling	Text + Log	Cybersecurity
Y. Liu et al. [87]	IIoT + VFL	Enabling	Text	Fault Diagnosis
Y. Xia et al. [88]	IIoT + FCL	Enabling	Sensor Data	Fault Diagnosis
Our	AIoT	Enabling	Flow Data	DDoS Detection

of Transformer combined with other machine learning models, activation functions and other common modules, and its weights are generated by training a large number of training data sets. Finally, perform tasks, including token classification, inference tasks, and prediction tasks. Analyzing LLM's core reveals that, despite its design for language processing, its ML-based backbone can fundamentally handle non-text flow data. Therefore, the key to LLM processing flow data is how to design a token generation module suitable for data to generate flow token and embedding parts.

The flow is regarded as a language to generate a flow token after tokenization. Through the context analysis ability of LLM, the correlation between the front and back flow tokens can be analyzed and the deep context features can be learned. This is difficult to achieve by conventional AI models. Conventional AI often detects attack flow only for the characteristics of a single flow, such as header characteristics, interval time, etc. This method ignores the important relationship between flows. DDoS attacks overwhelm hosts by denying access via voluminous, interrelated flows. After flow tokenization, LLM can not only mine the deep information between flow tokens, but also learn the correlation between each flow feature. Therefore, the analysis method using LLM is more in line with the principle of DDoS attack. Finally, the DDoS detection task is corresponding to the token classification task to realize the processing of flow data by LLM.

The pre-training purpose of Llama3 used in this paper is to predict the next token. Token classification task is not its native function, which needs to be realized by adding a linear classification layer to map to the target label space. Therefore, the same method is used in the flow token classification to achieve the final DDoS detection. In addition, since the requirement of DDoS detection does not need to predict the next token, the mask mechanism blocks the observable range of the subsequent input flow in order to ensure the feasibility of the prediction. In order to analyze all the input flow more widely, the mask mechanism is deleted. The proposed reverse filling mechanism ensures that the integrity of the LLM architecture will not be destroyed after the mask removal mechanism, as shown in III B. Considering the difficulty of using a large amount of data to train LLM, the method proposed in this paper can keep the LLM backbone unchanged, and the idea of designing only token, embedding and linear layer modules is reasonable and feasible.

2) *DDoS Categories and Correlations*: In this paper, DDoS is divided into five categories: Botnet [11]: Botnet infects a large number of hosts to form a one-to-many network for bots to launch attacks. Application layer DDoS (AppDDoS) [89]: Puppet hosts use the application layer protocol to issue a large number of requests to the server to exhaust resources. Network Layer DDoS (NetDDoS) [90]: Depletes resources by repeatedly sending ACK or SYN packets without responding to a flooding attack. Low-rate DDoS (LDDoS) [91]: Slow data transmission prevents the server from interrupting the connection with the attacking host to exhaust resources. Reflex DDoS (DrDoS) [13]: Sends a large number of packets to the server by forging packets and IP addresses to overwhelm the target host. From the above five principles of DDoS attacks is not difficult to see, DDoS attacks often use duplicate protocols and often go through the same IP/Port address. This shows that DDoS attacks have similarities in the direction of flow, source address, destination address, protocol and other characteristics. Based on this similarity and combined with the ability of LLM context correlation, DDoS flow can be better detected. However, the routinely collected DDoS flow is sorted by default in chronological order. This sorting method is difficult to reflect the characteristics of DDoS attacks. Detection in this order weakens the LLM's attention to the related rows before and after the flow. In order to enable LLM to better play its contextual performance, this paper designs a order assignment module based on the correlation of DDoS attack flow.

III. OVERVIEW OF CE4LDD

This section mainly introduces the overall composition and principle of CE4LDD detection scheme in AIoT, including the detection architecture, cloud edge function and the working principle of each component. It elaborates how to use the correlation between flow and realize DDoS attack detection in different application scenarios based on the performance of LLM through the bottom-up cloud edge collaboration method. The key symbols and notations are listed in Table III.

A. Overall Architecture

The system architecture is shown in Fig. 1, including the cloud, edge service layer and device layer. In the device layer, the set of IoT devices accessed is represented as $\mathcal{D} = \{D_1, D_2, \dots, D_K\}$. For the k -th device $D_k \in \mathcal{D}$, it selects the

TABLE III
KEY SYMBOLS AND NOTATIONS

Notation	Description
$\mathcal{D}$	Set of IoT devices, where $\mathcal{D} = \{D_1, D_2, \dots, D_K\}$
$\mathcal{F}$	Set of flows from IoT devices, where $\mathcal{F} = \{F_1, F_2, \dots, F_K\}$
F_k	Set of flows for the i -th IoT device, where $F_k = \{f_{k1}, f_{k2}, \dots, f_{kN_k}\}$
$\mathcal{S}$	Set of edge servers, where $\mathcal{S} = \{S_1, S_2, \dots, S_m\}$
$\mathcal{M}$	Set of edge models, where $\mathcal{M} = \{M_1, M_2, \dots, M_m\}$
$\mathcal{C}$	Set of edge clusters, where $\mathcal{C} = \{C_1, C_2, \dots, C_P\}$
$\mathcal{G}$	Set of token generation groups, where $\mathcal{G} = \{G_1, \dots, G_P\}$
B_u^k	The u -th regional flow block obtained by sorting the flows of the k -th IoT device.
$Seq(\cdot)$	The function for assigning order to the flows.
$g_i(\cdot)$	The selection weight function of the i -th token generation group.
y^{Token}	The output of flow tokenization.
X_k^{Token}	The tokenized output of all flows for the k -th IoT device.
X_k^{Em}	The output of the embedding.
y^{Bb}	The output of the LLM backbone.
y^{Class}	The result of linear classification.
$\mathcal{L}_1$	The loss function for training the embedding and linear projection layers.
$\mathcal{L}_2$	The loss function for training the token group selection gating mechanism
τ	The trust threshold of the detection results.

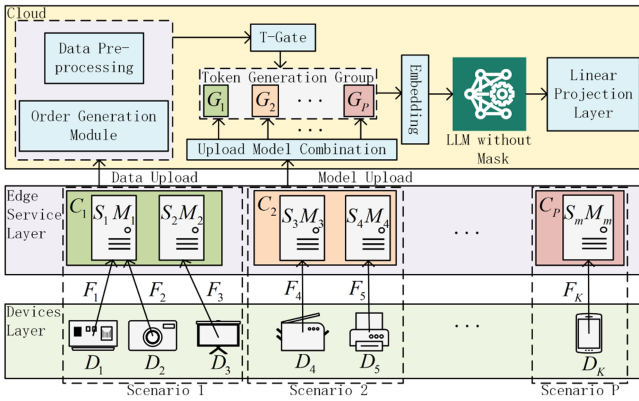


Fig. 1. CE4LDD schematic diagram in AIOT.

edge server it connects according to the nearest principle and transmits the flow data $F_k = \{f_{k1}, f_{k2}, \dots, f_{kN_k}\}$ to it, where $f_{kn} \in F_k$ represents each flow data collected by the device, and N_k represents the number of flows in the k -th device. Each flow is represented by the feature $f_{kn} \in \mathbb{R}^{1 \times B}$ obtained by the edge, where B is the number of features. Because the analysis using all the features of the flow is too burdensome for communication and does not help much for performance prompts, in the selection of the number of features, feature selection methods can be used, such as random forests, to select some important typical features for analysis [92]. The set of all flow data is represented as $\mathcal{F} = \{F_1, F_2, \dots, F_K\}$. In the edge service layer, the edge server set is represented as $\mathcal{S} = \{S_1, S_2, \dots, S_m\}$, and one neural network model M_i is deployed in each edge server S_i . The set of models is represented as $\mathcal{M} = \{M_1, M_2, \dots, M_m\}$, exists $|\mathcal{S}| = |\mathcal{M}| = m$. Each edge server connects to the wireless transmission node according to the proximity principle, which is responsible for transmitting the data and models in the edge server to the cloud. All edge servers connected to the same node form an edge server cluster, denoted as $\mathcal{C} = \{C_1, C_2, \dots, C_P\} =$

$\{\{S_1, \dots, S_i\}_1, \dots, \{S_j, \dots, S_m\}_P\}$, where $S_i, S_j \in C_x \subseteq \mathcal{S}$. The clusters in the edge service layer and the devices connected to the corresponding clusters in the device layer constitute a multi-application scenario of the IoT, a total of P scenarios.

The composition in the cloud of CE4LDD is shown in Fig. 1, which mainly includes two parts: pre-processing and LLM detection. The pre-processing component handles data and models from the edge, encompassing the data pre-processing module, the order generation module, T-Gate and the upload model merging module. The data pre-processing module is responsible for the preliminary preprocessing of the flow data uploaded from the edge data to the cloud. The order generation module is responsible for sorting the preprocessed flow data according to its characteristics, giving it order, so as to facilitate the subsequent LLM analysis of the correlation between flow. T-Gate select the corresponding TGG by analyzing the type of attack that the flow data may belong to. The upload model merging module divides the uploaded model according to the edge server cluster, deletes the final output layer and averages the output of the last hidden layer.

The LLM detection part is responsible for classifying the processed flow to distinguish whether it is DDoS attack flow, which is composed of TGG, embedding module, LLM without mask and linear projection layer. TGG is composed of a neural network model uploaded by the edge server, which is responsible for tokenizing the flow data and performing high-dimensional mapping through the embedding module to facilitate input to LLM for processing. The model in each edge server cluster deletes the output layer and combines them to form a TGG, which can be represented as $\mathcal{G} = \{G_1, G_2, \dots, G_P\}$, where $G_x \subseteq \mathcal{G}$. The LLM without mask mechanism is responsible for processing the data after embedding. Through the mask-less mechanism, it focuses on a wider range of flow and pays attention to the correlation between its flow, so as to facilitate the token classification task of the tokenized flow token to achieve the distinction of DDoS attack flow. The linear projection layer is responsible for the dimensionality reduction mapping of the high-dimensional data output by the LLM and the final classification result is output.

B. Function Description and Implementation

This section mainly describes the functions of each module in the edge and cloud and the basic principles of its implementation. It describes in detail how to use LLM to process flow data and cooperate with the detection ability of the edge model to realize the detection of DDoS attack flow, including flow pre-processing, order assignment, tokenization and embedding, LLM detection and final result output.

1) *Function Description and Implementation in Edge:* The edge application scenario consists of two parts: the IoT device layer and the edge service layer. IoT devices are mainly responsible for collecting flow in the network and transmitting it to edge servers for processing. The edge server includes two functions: flow feature extraction and edge attack detection model training. For the flow transmitted by the device, the characteristics of the flow is used to represent the flow. Firstly, feature extraction is carried out, and important features are screened by machine learning and other methods. Each flow is represented by B features as $f_{kn} \in \mathbb{R}^{1 \times B}$. Then, personalized model training is performed based on the collected flow and filtered features. The detection model obtained by has a certain detection preference based on

the flow data obtained by the edge server cluster. For example, the AppDDoS attacks in this application scenario are more, and the trained model has a strong detection ability for AppDDoS, which is called edge personalization model. For the obtained i -th edge personalization model $M_i \in \mathcal{M}$, it can be expressed as $M_i = \{\mathcal{I}_i, \mathcal{L}_i, \mathcal{O}_i\} = \{\mathcal{I}_i, (L_{i1}, N_{i1}), \dots, (L_{iq}, N_{iq}), \mathcal{O}_i\}$, where $\mathcal{I}_i$ represents the input layer set, $\mathcal{L}_i$ represents the hidden layer set and $\mathcal{O}_i$ represents the output layer set. The (L_{iq}, N_{iq}) represents the q -th hidden layer in the model M_i and the number of neurons contained in the hidden layer is N_{iq} , while the total number of hidden layers in the model is q_i . Note that in order to ensure that the size of the data dimension intercepted from the model is consistent, it is necessary to make the last hidden layer of the model in the same edge cluster have the same number of neurons N_{iq_i} . After obtaining the characteristics of the flow and the edge detection model, the wireless transmission node is responsible for transmitting the two to the cloud for subsequent processing. In addition, according to the above model set, the TGG representation can be updated to $\mathcal{G} = \{G_1, \dots, G_P\} = \{\{\mathcal{I}_1, \mathcal{L}_1\}, \dots, \{\mathcal{I}_i, \mathcal{L}_i\}_1, \dots, \{\dots\}_P\}$, where $\{\mathcal{I}_i, \mathcal{L}_i\} \in M_i \in \mathcal{G}_x \subseteq \mathcal{G}$. Finally, according to the detection results, the data and models of different scenes in the edge are uploaded to the cloud (see Section IV for details) and further detected by CE4LDD.

2) *The Function and Implementation of CE4LDD in Cloud:* The cloud function of CE4LDD includes two parts: pre-processing and LLM detection. The pre-processing part is responsible for processing and selecting the data and models uploaded from the edge to the cloud. The data pre-processing module and the order generation module are responsible for processing the flow data. The upload model merging module is responsible for processing the edge models of different scenarios. T-Gate is responsible for matching data and TGG; the LLM detection part is jointly coordinated by the TGG, the embedding module, the LLM without mask and the linear projection layer to realize the detection of DDoS flow.

Data pre-processing module: Processing flow data uploaded by different scenarios. Since the flow data transmitted from the edge has been preprocessed by feature extraction and screening before it is used for edge model training and testing. In this process, it is necessary to filter and normalize the transmitted features so that the model can quickly converge during subsequent training. Assume that $f_{kn} \in \mathbb{R}^{1 \times B}$ is the flow transmitted to and $\bar{f}_{kn}$ represents the average value of flow features and $\hat{f}_{kn}$ represents the variance. The normalized f_{kn}^s can be expressed as

$$f_{kn}^s = \frac{|f - \bar{f}_{kn}|}{\sqrt{\hat{f}_{kn} + \alpha}}. \quad (1)$$

Where α is the minimum value, in order to prevent the denominator becoming 0. In the course of the experiment, a random minimum value can be used to set α . Because for $\hat{f}_{kn} = 0$, $\exists |f - \bar{f}_{kn}| = 0$, it cannot affect the actual results.

Flow order generation module: For the flow data after processing, in order to use LLM to analyze the correlation between flow, it is necessary to give a clear order to the flow. Because the collection of flow is carried out in chronological order and the order of flow in the time dimension is not obvious, only focusing on the time dimension is not enough to reveal the internal relationship and characteristics of the flow itself.

Based on the principle of DDoS attack, the flow order hides the attacker's attack mode. When there is frequent flow through the same IP address or the same port, it is likely to be a harbinger of malicious attacks. Similarly, the high frequency use of specific protocols may also represent the occurrence of attacks. These attacks indicate that the starting position, destination, transmission direction and mode of flow can be used as an important basis for distinguishing flow categories. Five-tuple $\langle IP_{src}, IP_{dst}, Port_{src}, Port_{dst}, Protocol \rangle$ is used to divide and sort flow F_k according to source IP, destination IP, source port, destination port and protocol.

The flow of the same source and purpose is divided into a regional flow block, which is represented as B_u^k , where u is the index, and the divided B_u^k is sorted according to the transmission protocol used to sort the regional flow blocks of the same protocol together.

$$\begin{aligned} B_u^k &= \{f_{kn} | f_{kn} \in f_{ki,kj}, \\ (Src(f_{ki}) &= Src(f_{kj}) \cap Dst(f_{ki}) = Dst(f_{kj})) \cup \\ (Src(f_{ki}) &= Dst(f_{kj}) \cap Dst(f_{ki}) = Src(f_{kj}))\}. \end{aligned} \quad (2)$$

where f_{ki}, f_{kj} represent the i, j -th flow in the k -th device. $Src(\cdot)$ represents the source of flow, and $Dst(\cdot)$ represents the purpose of flow. For the divided flow block B_u^k , its interior can be divided more carefully according to the direction of flow transmission. Assuming that B_{SD}^k represents the source-to-destination flow and B_{DS}^k represents the destination-to-source flow, it can be expressed as follows.

$$\begin{aligned} B_{SD}^k &= \{f_{kn} | f_{kn} \in f_{ki,kj}, Src(f_{ki}) = Src(f_{kj})\}, \\ B_{DS}^k &= \{f_{kn} | f_{kn} \in f_{ki,kj}, Dst(f_{ki}) = Dst(f_{kj})\}. \end{aligned} \quad (3)$$

Therefore, B_u^k can be expressed as

$$B_u^k = \{B_{SD}^k, B_{DS}^k\}. \quad (4)$$

$|B_u^k|$ is defined as the number of flows it contains. The function $Seq(\cdot)$ is used to represent the order assignment process of the entire generated regional flow block. It should be noted that the $Seq(\cdot)$ function not only contains the generation of regional flow blocks, but also is responsible for sorting the generated blocks according to the protocol. For the flow data F_K of the k -th device, the flow can be expressed as equation (5) after being order assigned.

$$Seq(F_k) = \{B_1^k, B_2^k, \dots, B_U^k\}, \sum_{u=1}^U |B_u^k| = N_k. \quad (5)$$

Through the generation method of regional flow block, the transmission direction and mode of flow, the starting point and end point position can be included in the subsequent sorting basis, so as to realize a more reasonable sorting of flow to mine more correlation between flow.

After generating regional flow blocks according to the above sorting method, in order to more effectively input into LLM and make full use of its correlation detection ability and parallel processing characteristics, the order matrix of regional flow blocks is generated and the generation diagram of the order matrix is shown in Fig. 2.

The obtained flow blocks are sorted by rows to obtain the flow order matrix. For N_k flow data, assuming that the number of flows per row is V , the number of rows required can be expressed as $W = \lceil \frac{N_k}{V} \rceil$. The input flow is filled sequentially until the last flow. For the last line, if it is not filled completely, the input flow is filled backward until it is filled. This sorting method can

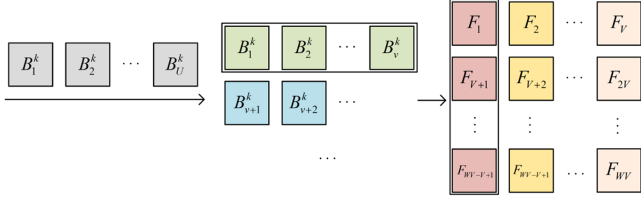


Fig. 2. Schematic diagram of flow order matrix generation.

ensure that the number of flows in each row of the generated flow matrix is the same, and the correlation between the front and back reaches the maximum. For the subsequent input is processed in LLM, the input is performed by column input. Because when generating regional flow blocks, the flow with strong correlation is clustered and sorted, and this order is maintained when the flow order matrix is generated, which ensures that each flow has a strong correlation with the flows at both ends, so as to give full play to the ability of LLM to extract correlation. Moreover, because the length of a DDos flow is variable, we represent flow using a fixed set of flow characteristics so that the token length remains controllable. In other words, the token length can be adjusted by selecting the number of flow characteristics.

For the generated flow order matrix, its row index r and column index c can be expressed as

$$r = \begin{cases} \lfloor \frac{i}{V} \rfloor + 1, i \bmod V \neq 0 \\ \frac{i}{V}, i \bmod V = 0 \end{cases},$$

$$c = \begin{cases} i \bmod V, i \bmod V \neq 0 \\ m, i \bmod V = 0 \end{cases}. \quad (6)$$

where i represents the order of flow in $Seq(F_k)$ after sorting. So far, the flow order is given to the process to complete, and the change of the disordered flow to the ordered matrix $F_k \rightarrow X_k$ is realized, where $F_k \in \mathbb{R}^{N_k \times B}$, $X_k \in \mathbb{R}^{W \times V \times B}$. All subsequent flow processing processes are based on this order, and the input is entered into the LLM by column input to ensure that the LLM can fully extract the correlation between the successively input data.

T-Gate: Tokenization is required before inputting LLM for the flow given order. In the scheme proposed in this paper, the work of tokenization is undertaken by TGG, and how to select the appropriate group is responsible by T-Gate. This design is inspired by Mixed Expert Models (MoE) [93]. The gating uses a small neural network to obtain the probability score of the attack type of the input flow according to the flow characteristics, and calculates the activation weight of each TGG according to the output probability score.

For the edges where there are P edge model clusters, P TGGs are generated according to the model uploaded by them, which correspond to P personalized attack types, denoted as Att_p , so gate control generates P weights, expressed as $\{g_1 \cdots g_P\}$. For the input flow f_{kn}^s , the weight is expressed as follows

$$g_i(f_{kn}^s) = \text{softmax}(W_g f_{kn}^s + b_g), i = 1 \cdots P \quad (7)$$

For the $f_{kn}^s \in Att_p$ of the p -th attack type, $g_p(f_{kn}^s) \rightarrow 1$ and $\{g_1(f_{kn}^s), \dots, g_i(f_{kn}^s), \dots, g_P(f_{kn}^s)\} \rightarrow 0, i \neq p$ are required, indicating that the more accurate the classification of the attack, the more reasonable the selection of the gating. Because of $\sum_{i=1}^P g_i(f_{kn}^s) = 1$, the weight entropy $H(g_i(f_{kn}^s))$ can be used to reflect the performance of the selection gating, $H(g_i(f_{kn}^s))$

can be expressed as

$$H(g_i(f_{kn}^s)) = - \sum_{i=1}^P g_i(f_{kn}^s) \cdot \log(g_i(f_{kn}^s)). \quad (8)$$

The closer the entropy value is to 0, which means $H(g(f_{kn}^s)) \rightarrow 0$, the better the effect of selection gating. For the trained gating model, the maximum weight of the output is selected as the basis for the selection of TGG, which can be expressed as $\{G_p | g_p(f_{kn}^s) = \max(g(f_{kn}^s))\}$.

The upload model is merged to generate TGG and embedding: After the gated control makes a decision, the flow data is transmitted to the specified TGG G_p , which is composed of the models in the corresponding edge server cluster, and the transmission process is based on the EAL-BT architecture.

After the edge model is transmitted to the cloud, the upload model merging module merges the models of different scenarios to generate TGG. For model $M_i = \{\mathcal{I}_i, \mathcal{L}_i, \mathcal{O}_i\}$, the input layer $\mathcal{I}_i$ and the hidden layer $\mathcal{L}_i$ of the model are saved, the output layer $\mathcal{O}_i$ is discarded, and all the connections and weight relationships between the input layer and the hidden layer and the hidden layer are retained, and the detection ability of the edge model is retained to the greatest extent. The flow data input is a high-dimensional vector through the output of the last hidden layer. Assuming that the input flow data is $f_{kn} \in \mathbb{R}^{1 \times B}$, since the edge model is frozen in the subsequent process, $M_i(\cdot)$ is used to represent the output of the last hidden layer of the model, which means that $y_i = M_i(f_{kn})$, where $y_i \in \mathbb{R}^{1 \times N_{iq_i}}$, $W^{L_{iq}} \in \mathbb{R}^{B \times N_{iq_i}}$, $W^{L_{iq}}$ is the weight matrix of the last hidden layer.

For the edge server cluster $C_p \in \mathcal{C}$, the number of models it contains is $|C_p|$. As the last hidden layer of each model is configured with the same number of neurons, leading to identical output dimensions, thus $|y_1| = |y_2| = \dots = |y_{|C_p|}|$ can be obtained. So the output of TGG can be expressed as

$$y^{Token} = \left[\left[\frac{\sum_{i=1}^{|C_p|} y_i}{|C_p|} \right] \right] \in \mathbb{R}^{1 \times N_{iq_i}}. \quad (9)$$

Where $\lceil \cdot \rceil$ is defined as a rounded symbol. Therefore, the output of all the flow of the k -th device after TGG is expressed as $X_k \rightarrow X_k^{Token} \in \mathbb{R}^{W \times V \times N_{iq_i}}$, which indicates that the dimension of the feature is mapped from B dimension to N_{iq_i} , which is the number of neurons in the last hidden layer.

For the generated flow token matrix, in order to facilitate batch processing and ensure that the order length is consistent, in the subsequent input to LLM according to the column, the direction filling processing in the order matrix generation process ensures that the order length of this input method is consistent. In order to facilitate the description, the subsequent processing of the token matrix is still introduced from the perspective of the matrix.

After obtaining the output of the TGG, it needs to be mapped in high dimension to realize embedding, which is convenient for subsequent input into LLM. For the output of TGG, its feature extraction ability comes from the edge model. In order to use the performance of LLM to detect malicious flow, the linear classification results after LLM output are used to train the embedding model.

For the flow data that the order is given, in order to make full use of its order, the MLP-Mixer composed of Token-Mixing MLP and Channel-Mixing MLP is used for high-dimensional mapping for embedding. Compared with conventional MLP, MLP-Mixer processes data channels and spatial dimensions through two independent MLPs, which can better capture global

features and the relationship between features, and is more suitable for processing flow data after giving order.

For the tokenized flow data X_k^{Token} , the data is first flattened to $X_{k,flatten}^{Token} = [x_1, x_2, \dots, x_{W \times V}]$, and the flattening data is linearly transformed by the weight coefficient W^{TM} to $\hat{X}_{k,flatten}^{Token} = W^{TM} X_{k,flatten}^{Token}$, where $\hat{X}_{k,flatten}^{Token} \in \mathbb{R}^{W \times V \times B}$.

Finally, the output $X_k^{TM} = X_{k,flatten}^{Token} + \sigma(\hat{X}_{k,flatten}^{Token})$ of the Token-Mixing MLP is obtained by adding the original input and the linear change after activation through the residual connection, where $\sigma(\cdot)$ represents the nonlinear activation function (such as GELU, etc.). The X_k^{TM} is input into the Channel-Mixing MLP, and the feature fusion transformation is performed on it by the weight coefficient W^{CM} to obtain $\hat{X}_k^{CM} = W^{CM} X_k^{TM} \in \mathbb{R}^{W \times V \times B}$. The output $X_k^{CM} = X_k^{TM} + \sigma(\hat{X}_k^{CM})$ of Channel-Mixing MLP is obtained by residual connection. The high-dimensional mapping weight matrix W^h is used to map it to high dimensions to obtain

$$X_k^{Em} = W^h X_k^{CM} \in \mathbb{R}^{W \times V \times H}. \quad (10)$$

Where H denotes the high-dimensional dimension to be mapped.

LLM without mask: The flow data is mapped into high-dimensional data by embedding and input into LLM. Since it remains order and in the form of matrix, considering the characteristics of LLM processing input data, it is input by column. In LLM, in order to ensure the reliability of the output prediction results, a mask mechanism is introduced to block all data after the current input, so as to prevent the input data observed in advance from affecting the prediction results. However, in the input process of flow data, the purpose is to analyze the relationship between flow tokens to help detect DDoS flow without predicting the next flow token. Therefore, the existence of the mask mechanism affects the LLM's analysis of the correlation between all input flow. In the proposed flow token generation process, padding tokens are not used to unify the length of the input flow, but the generated flow matrix is input column by column to achieve length unification. Therefore, the mask mechanism can be ignored and its batch processing ability and model performance can not be affected. The way to ignore the mask mechanism is to directly return the causal mask of the lower triangle generated in the `_prepare_decoder_attention_mask` to None, so as to realize the full connection of attention to observe all the input flow token, and improve the ability of the model to analyze the internal relationship of the context flow token.

For the flow data after the order is given, it still maintains its order after TGG and embedding. In order to enable LLM to pay attention to the relationship between a wider range of flow and analyze the order relationship between flows, make full use of its order, LLM also needs to ignore the mask mechanism. The LLM ignoring the mask mechanism can focus on all the data before and after the current input. The improved LLM structure diagram for flow correlation analysis is shown in Fig. 3.

The improved LLM structure is based on llama3 as the backbone, which consists of flow processing part, backbone part and classification task part. Flow Processing maps the flow to high dimensions through the token generation team and embedding to get X^{Em} input to backbone. After normalization in backbone, we first analyze the relationship between flow by the attention mechanism without adding padding token to bypass the mask.

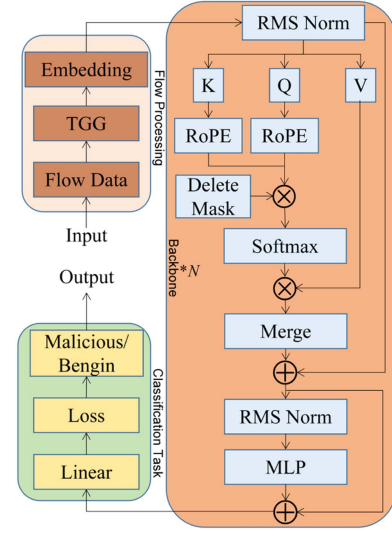


Fig. 3. LLM structure diagram for correlation analysis.

After a series of operations such as merging and normalization, the feature is input into a MLP for high-dimensional extraction. After repeatedly passing through the N layer backbone, the output result is expressed as $y^{Bb} = \text{Backbone}(X^{Em})$. The input X^{Em} of the LLM is input by column, and then the flow token after high-dimensional mapping is classified according to the idea of token classification in llama3. Finally, enter the classification task part, perform dimensionality reduction through the linear layer and output the final classification result $y^{Class} = \text{Class}(y^{Bb})$ in combination with the loss function. The effective part is the first $|F_k|$ term of y^{Class} , and the last $W \times V - |F_k|$ term is the reverse filling of the complete matrix in order to supplement the order assignment, rather than the original flow data.

Corresponding the entire flow detection process to the LLM workflow can better understand how the TGG, the embedding module, the LLM without mask, and the linear projection layer work together. The text input in LLM is divided into tokens and vectorized embedding into the LLM backbone. The proposed scheme divides the flow order matrix into flow tokens and vectorizes them through the TGG. The TGG is composed of an edge model with characteristic DDoS attack detection capabilities, and then the flow token is mapped to a high-dimension through the embedding module and input into the LLM backbone. The DDoS detection task is corresponding to the token classification task in LLM, and the flow token is classified. For the output that is still in a high-dimensional state after classification, the linear projection layer is used to project it to low-dimensional to obtain the final DDoS detection result. During the training process, the LLM backbone is frozen, and the embedding and linear projection layers are considered as a whole for joint training. We exclude the LLM backbone from the training scope because fine-tuning it requires substantial computational resources, the available dataset is far smaller than the corpus used during the LLM's pretraining, and fine-tuning risks degrading the model's pre-trained contextual understanding. Furthermore, fine-tuning offers limited opportunity to leverage the capabilities of the edge model.

So far, the whole CE4LDD scheme has been completed. In AIoT, through cloud edge collaboration, the detection ability

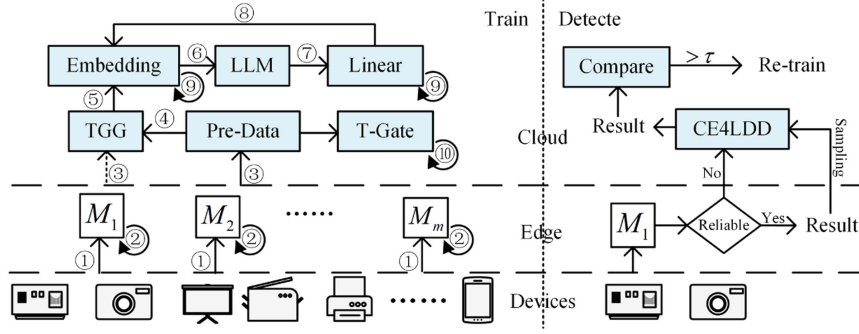


Fig. 4. The complete EAL-BT training and testing process.

of the edge model is aggregated from different edge scenarios from bottom to top to construct the corresponding TGG, and the TGG is selected based on weight entropy by selection gating. The flow data is transmitted from the edge to the cloud. After pre-processing, it is divided into blocks and sorted by the order generation module according to the given sorting rules to generate a flow order matrix. Finally, the MLP-Mixer is used to embed into a high-dimensional space, combined with the LLM bypassing the mask and the linear projection layer and finally the detection result is output. A bottom-up DDoS flow detection scheme using the correlation between flow is implemented.

IV. THE TRAINING AND TESTING PROCESSES OF EAL-BT

In order to apply to AIoT scenarios and make targeted detection of different DDoS attacks in different AIoT application scenarios, this paper presents a bottom-up training and detection process called EAL-BT. The detection process also includes two parts: routine detection and supervision detection. Through EAL-BT, the CE4 LDD in the cloud is assisted by the detection model in different scenarios of the edge. The complete training and detection process is shown in Fig. 4, which includes the transmission direction of data and model. The left side is the training process and the right side is the detection process. In order to facilitate the description, the detection process is only described for a simple situation with only one scene and only one edge model.

For the training phase, four parts need to be trained, including the edge scenarios detection model, the T-Gate, the MLP-Mixer for high-dimensional mapping and the linear projection layer. For the conventional detection stage, it is mainly composed of edge detection and cloud detection. The edge detection is responsible for the flow in the edge; cloud detection is responsible for the flow flowing through the cloud and the unconvinced flow in the edge. Supervised detection is to randomly sample the credible detection results, combine the detection results of the edge and the cloud to ensure the detection ability of the model, and can timely discover the deficiencies and update the model.

A. The Training Process of EAL-BT

The training process is shown on the left side ①-⑩ in Fig. 4.

Step ① represents the transmission of flow data $\mathcal{F}$ from the accessed IoT device $\mathcal{D}$ to the edge server $\mathcal{S}$.

Step ② indicates that the edge model $\mathcal{M}$ used to detect malicious flow is trained according to the collected data and the characteristics of flow data are extracted and screened.

Step ③ means that the flow data and the trained model are transmitted to the cloud for data preprocessing and TGG generation for subsequent processing. The data preprocessing module processes the received data $\mathcal{F}$, the order generation module blocks the flow and generates a flow order matrix X_k , and the upload model joint module jointly generates the TGG according to the server cluster. In the training phase, since the TGG does not have a preference for dealing with malicious flow at this time, it does not need to go through the T-Gate for TGG selection, and only needs to input the data to the TGG formed by the same server cluster. Step ④ indicates that the generated flow order matrix is input to the corresponding TGG.

Step ⑤⑥⑦⑧⑨: Firstly, the output of TGG is mapped to high-dimension by embedding and then input to LLM without mask. The final detection result is obtained by reducing the dimension of LLM output through linear projection layer. Based on the input flow matrix X_k and the output detection result y^{Class} , the embedding module composed of MLP-Mixer and the linear projection layer are jointly trained. The loss function can be expressed as:

$$\mathcal{L}_1 = -\frac{1}{|F_k|} \cdot \left[\sum_{r=1}^{W-1} \sum_{c=1}^V (y_{r,c}^{true} \log y_{r,c}^{Class} + (1-y_{r,c}^{true}) \log(1-y_{r,c}^{Class})) + \sum_{c=1}^{V-WV+|F_k|} (y_{W,c}^{true} \log y_{W,c}^{Class} + (1-y_{W,c}^{true}) \log(1-y_{W,c}^{Class})) \right]. \quad (11)$$

Where $y_{r,c}^{true} \in X_k$, r, c is the matrix index, which represents the true label of the original data, $y_{r,c}^{Class}$ represents the detection result, for the last row of the matrix data, the last $WV - |F_k|$ item is the reverse filling data, so only the first $V - WV + |F_k|$ item is considered to avoid repeated calculation.

After training the embedding and linear projection layer, enter step ⑩ to train the T-Gate. For flow data, add a label $y^{Cluster} \in \mathbb{R}^{1 \times P}$ to it to represent the edge server cluster of its source. For the selection weight of the T-Gate output, the cross entropy loss is used to calculate its matching degree with the real label as follows:

$$\mathcal{L}' = -\sum_{i=1}^P y_i^{Cluster} \log(g_i). \quad (12)$$

In order to ensure the reliability of the weight generation, the cross loss entropy $H(g_i)$ is used to control the dispersion of the

weight. The smaller the entropy value is, the more reliable the selection is. Therefore, the total loss function can be expressed as

$$\begin{aligned}\mathcal{L}_2 &= \mathcal{L}' + \lambda H(g_i) \\ &= -\sum_{i=1}^P y_i^{Cluster} \log(g_i) + \lambda \left(-\sum_{i=1}^P g_i(f_{kn}^s) \cdot \log(g_i(f_{kn}^s)) \right).\end{aligned}\quad (13)$$

B. The Testing Process of EAL-BT

The conventional detection process is shown on the right side of Fig. 4, including edge detection and cloud detection.

Firstly, the IoT device transmits the flow data flowing through it to the edge server, and then uses the detection model $\mathcal{M}$ trained by the edge server to perform edge detection and obtain the detection results. At this time, it can be divided into two cases: the detection results of the trust edge model and the detection results of the distrust edge model. It is assumed that the probability of the model judging the flow as malicious flow is P , and the probability of normal flow is $1 - P$. When one of them is closer to 1, which means $P \rightarrow 1$ or $1 - P \rightarrow 1$, the test results are more credible. On the contrary, $P, 1 - P \rightarrow 0.5$ indicates that the model cannot accurately classify flow. In order to better reflect the credibility of the detection results, entropy is used to represent the credibility, which is represented as

$$H_C = -(P \log(P) + (1 - P) \log(1 - P)). \quad (14)$$

For P tends to 1 or 0, $H_C \rightarrow 0$, the credibility of the prediction results is high; $P \rightarrow 0.5$, the entropy value tends to the maximum value $H_C \rightarrow \log(2)$ and the uncertainty is high. The credibility threshold is set to τ , for $H_C < \tau$, the detection result is credible, otherwise the $H_C > \tau$ indicates that the result is not credible. For trusted results, the detection results are output directly; for untrusted results, the flow data and results are transmitted to the cloud together and further detected.

Firstly, the flow data is preprocessed and given the order to generate the flow order matrix. Through T-Gate, select its suitable TGG. The flow token is generated by the selected TGG. Then input it to the LLM without mask by embedding mapping to high-dimensional and the output result is linearly projected to obtain the final detection result. For untrusted detection results, if the detection results in the edge and the cloud are inconsistent, the detection results in the cloud shall prevail. The inconsistent results are recorded. Finally, the detection results are returned to the edge server and the IoT device to complete the entire detection phase process.

In addition, in order to ensure the reliability of the detection performance of the detection system, and to update the new attack in time, the supervision and detection process is introduced to supervise the whole system. In the conventional detection process, the untrusted detection results are transmitted to the cloud to improve the credibility. However, for the credible results that do not need to be transmitted to the cloud, there may still be detection errors. Therefore, for the detection results that are judged to be credible, a random sampling method is adopted to randomly send the flow data to the cloud for detection, and the detection results of the two are compared. In the case of consistent detection results, no processing is performed, and the results of inconsistent detection results are based on the CE4LDD results in the cloud. In the joint detection stage, the data whose edge untrusted results are inconsistent with the

detection results in the cloud are collected and all the results with objections are summarized and recorded. If the number of recorded objection results exceeds the set threshold τ , it means that there may be unknown attacks or errors in the system. It is necessary to retrain the model and update the trained data set to adapt to new attacks and ensure the reliability of the detection results.

Latency is a primary concern in cloud-edge collaboration; therefore, we analyze the testing-process latency T . Detection latency is decomposed into three components: the edge latency T_{edge} , which includes edge-model inference time and gating-model computation time; the upload latency T_{upload} , which comprises data upload time and protocol overhead; and the cloud latency T_{cloud} , corresponding to the time the LLM requires to analyze DDoS flows. Consequently, the total detection latency has two cases. If the edge detection is trusted (i.e., $H_C < \tau$), then $T = T_{edge}$. If the edge is untrusted (i.e., $H_C \geq \tau$), then $T = T_{edge} + T_{upload} + T_{cloud}$. Thus, the latency of the proposed cloud-edge collaborative architecture is primarily governed by the trust threshold τ , other components are determined by model and system configurations, such as batch size and GPU performance.

V. PERFORMANCE EVALUATION

This section consists of two parts: experimental environment setting and performance comparison. The proposed system model is built and the performance is evaluated by simulating DDoS attacks on five different AIoT scenarios. Mainly explain the following three questions:

- Does the detection system based on LLM perform excellent in detection performance?
- Can flow order affect the detection performance of the model?
- Does the bottom-up of edge models in different scenarios help to improve performance?

A. Experiment Setting

For the overall architecture of the system, its important parameters are shown in Table IV. Five edge server clusters C_1, C_2, C_3, C_4, C_5 are set up to correspond to different DDoS attacks in five different AIoT scenarios, namely DrDoS, LDDoS, NetDDoS, AppDDoS and BotNet. In the five edge server clusters, five different models are trained based on the DDoS flow data in the cluster. Three identical edge models are deployed in each cluster, and the specific structure of each model is described in detail.

Dataset: The self-data¹ set is constructed by simulating five major types of DDoS attacks, and in order to avoid the influence of the deviation of the self-data set, the public CIC2019 DDoS data set [94] is used as a supplement, and the proportion of flow data from the two data sets is guaranteed to be close to 1 : 1. The construction method of self-data is as follows : multiple Ubuntu 18.04 virtual machines are deployed on the VMware vSphere platform, each of which is allocated with 8-core CPU, 16 GB memory, and 32 GB hard disk to form a three-layer network topology of attack domain-entrance routing-target domain to simulate attack flow in real scenarios. **LDDoS:** Of the 8 hosts, 4 attackers launched slow headers, slow body, shrew, slow read

¹DDoS Attack Datasets https://github.com/liliMpro/source_dataset

TABLE IV
IMPORTANT PARAMETERS OF EXPERIMENTAL DESIGN

Cluster	Edge Model	Attack Type	Flows
C_1	MLP Input: [Features, 12] Dense: [ReLU, 56] → [ReLU, 128] Output: [Sigmoid, 1]	DrDoS (DNS)	10000 (CIC2019)
		DrDoS (NTP)	10000 (CIC2019)
		DrDoS (MSSQL)	10000 (CIC2019)
		DrDoS (TFTP)	10000 (CIC2019)
		DrDoS (Memcached)	9000 (Our)
C_2	CNN Input: [12] Conv: [32,3,ReLU] → MaxPooling [2] Conv: [64,3,ReLU] → MaxPooling [2] Conv: [128,3,ReLU] → MaxPooling [2] Dense: [ReLU, 512] Output: [Sigmoid, 1]	LDDoS (UDPLag)	30000 (CIC2019)
		LDDoS (SlowBody)	3500 (Our)
		LDDoS (Shrew)	3500 (Our)
		LDDoS (SlowHeaders)	3500 (Our)
		LDDoS (SlowRead)	3500 (Our)
C_3	MLP-Mixer Input: [Features, 12] Dense: [ReLU, 256] → [ReLU, 256] → Flatten Output: [Sigmoid, 1]	NetDDoS (SYN Flood)	16000 (CIC2019)
		NetDDoS (UDP Flood)	16000 (CIC2019)
		NetDDoS (ACK)	10000 (Our)
C_4	LSTM Input: [25, 12] Bi-LSTM: [Tanh, 64] Dense: [ReLU, 256] Output: [Sigmoid, 1]	AppDDoS (WebDDoS)	146 (CIC2019)
		AppDDoS (CC)	10000 (Our)
		AppDDoS (HTTP Get)	3434 (Our)
		AppDDoS (HTTP Flood)	10000 (Our)
		AppDDoS (HTTP Post)	10000 (Our)
C_5	Transformer Input: [12, 12] Dense: [ReLU, 128] → Transformer Block [128, 4, 2] Output: [Sigmoid, 1]	Botnet (Ares)	30000 (Our)
		Botnet (BYOB)	2926 (Our)
		Botnet (Mirai)	2251 (Our)
		Botnet (Zeus)	327 (Our)

and other attacks through SlowHTTPTest, and 2 Web servers received flow. **AppDDoS**: four attackers use Hulk, Webbench, GoldenEye and other tools to simulate HTTP-Flood, CC attack, HTTP-Post and HTTP-Get. **DrDoS**: An attacker uses Scapy to forge requests to two reflection servers, generating reflective flow such as Memcached, TFTP, Chargen, NTP, SNMP, SSDP, etc. **Botnet**: Four controlled zombie hosts communicate with four CC servers, and the attack vectors include Ares, BYOB, Zeus and Mirai. **NetDDoS**: 3 attackers use hping to launch SYN, ACK, UDP flooding attacks. **Benign**: Use Python socket to generate legitimate requests in simulated 5 G public service, smart home, and MTC scenarios, and mix them with normal flow in the CICIDS2017 [95] data set to enrich the diversity of background flow. Grab the full amount of PCAP through tcpdump on the entry route. The CICFlowmeter tool is used to convert PCAP into a flow table of packet statistical features.

The important parameters for training and testing are shown in Table V. The number of feature representations of flow is set to $B=12$. Through the random forest and Boruta method [96], three types of features are first obtained through Boruta: Confirmed, Tentative and Rejected. The Rejected class represents insignificant features, so only the first two categories are retained. The random forest is used to calculate the importance I on the retained features, and the mean μ and standard deviation σ of the importance are calculated. The importance of each feature is standardized to $I^s = I - \mu/\sigma$. In order to remove the weak features, only the $I^s > 1$ features are retained. Finally, a total of 12 features are retained to represent flow. In the process of generating the flow order matrix, the total number of rows is

TABLE V
IMPORTANT PARAMETERS FOR TRAINING AND TESTING

Parameter	Value
Number of features	12
Total rows of flow matrix	64
Learning rate for mapping + linear projection training	0.001
Learning rate for T-Gate training	0.01
Embedding dimension	4096

set to 64, and the number of rows that can be accommodated in each row is determined according to the number of flows. The advantage of this setting is to ensure that the size of each column input is the same and a fair comparison is made. The final dimension of the embedding module after high-dimensional mapping is set to 4096 dimensions, which corresponds to the embedding dimension in the llama3-7/8B model [97]. This can ensure that the output of our embedding can be directly input into the Embedding itself in llama3. If the dimensions do not correspond to each other, it is necessary to change the original embedding setting of llama3 to correspond to our embedding direction, and the parameters and weights of the model need to be retrained.

The accuracy rate, precision rate, recall rate and F1-score are selected as the evaluation indexes of model detection

TABLE VI
PERFORMANCE COMPARISON OF MODELS ON DIFFERENT ATTACK TYPES: PROPOSED SCHEME VS. FIVE MODELS

	DrDoS		LDDoS		NetDDoS		AppDDoS		Botnet	
	Our	MLP	Our	CNN	Our	MLP-Mixer	Our	LSTM	Our	Transformer
Acc	99.20	96.70	99.16	97.18	99.40	98.53	99.28	98.02	99.36	98.47
Pre	99.21	96.25	99.35	97.30	99.44	98.50	99.12	97.89	99.40	98.68
Rec	99.18	97.12	98.97	97.07	99.37	98.57	99.46	98.15	99.32	98.28
F1	99.20	96.68	99.16	97.18	99.40	98.53	99.29	97.99	99.36	98.48

performance. The calculation formula is as

$$\begin{aligned}
 Acc &= \frac{TP + TN}{TP + TN + FP + FN}, \\
 Pre &= \frac{TP}{TP + FN}, \\
 Rec &= \frac{TP}{TP + FP}, \\
 F1 &= \frac{2TP}{2TP + FP + FN}.
 \end{aligned} \quad (15)$$

Where TP represents true positive, TN represents true negative, FP represents false positive, and FN represents false negative. In the experiment, the attack flow is set as a positive sample and the benign flow is set as a negative sample. For all the following experimental results, they are the results of averaging after repeating ten times.

B. Comparison With Five ML Models and One LLM-Based Baseline

1) *Comparison of Performance With Five Models on a Balanced Dataset:* The ratio of benign flow to attack flow is set to 1 : 1 to construct a balanced data set. The proposed scheme is compared with five ML models. For the five models, the DDoS attack types in the corresponding cluster are first used for corresponding training, and only the attacks in the corresponding cluster are detected. It should be noted that the five models used for comparison do not directly use the edge model in the proposed scheme. The experimental results are shown in Table VI. It can be seen that for different DDoS attacks in different clusters, the performance of the proposed scheme is higher than that of the conventional model in four evaluation indexes, and the detection ability of the proposed model to attack flow and benign flow is basically the same. For the 1 : 1 attack flow and benign flow distribution, the detection difficulty is relatively simple, but the proposed scheme still has advantages in performance.

In order to better observe the classification performance of different models under the balanced data set, the high-dimensional data generated by the above model flow data is reduced based on t-SNE. The coordinates of all positive points and negative points are averaged. The results are shown in Fig. 5. The distance between the two points after dimensionality reduction is 236, which is much larger than 63 of MLP, 58 of CNN, 136 of MLP-Mixer, 61 of LSTM and 115 of Transformer. The farther the distance between the positive point and the negative point, the easier it is to distinguish the attack flow and the benign flow, and the classification performance is better. It can be seen that although the proposed model has no obvious advantage in the four detection performance due to the simple detection difficulty of the balanced data set, the distance between the positive and

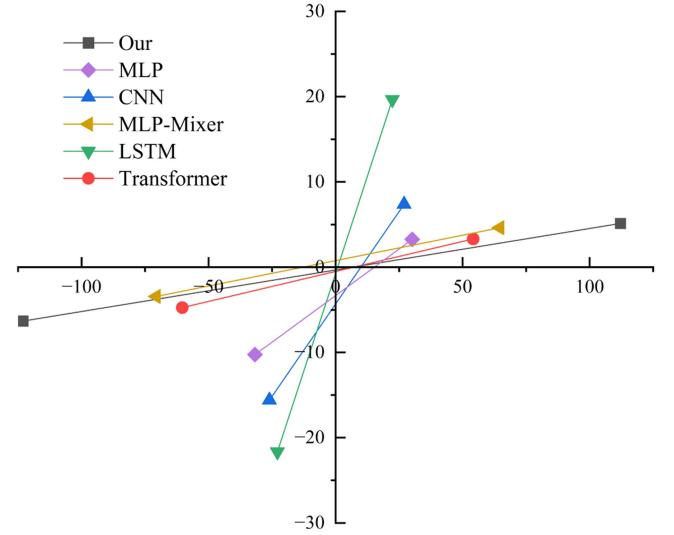


Fig. 5. Distances between positive and negative points after high-dimensional data dimensionality reduction on a balanced dataset: Proposed scheme vs. five models.

negative points is significantly larger than the other five models. It shows that the proposed scheme can learn more knowledge between flow data to better classify. The experimental results also provide an explanation for the excellent detection performance of the proposed scheme in subsequent experiments.

2) *Comparison of Performance With Five Models on an Imbalanced Attack Dataset:* Considering that the number of benign flow in the real network is far more than the number of attack flow, in order to better simulate the real flow situation. Modify the flow ratio column and set the attack flow to account for 10% of all flow to build an unbalanced data set. Comparing the performance of different methods on imbalanced data sets, the experimental results are shown in Fig. 6. The proposed scheme still maintains advantages in accuracy, recall and F1-score. Compared with the other five schemes, the average recall rate increased by 15.62%. The F1-score of the key index of the detection performance on the unbalanced data set is increased by 5.95% on average. Because the number of attack flow in unbalanced data sets is small, it is difficult for conventional methods to fully learn from limited data. In addition to the characteristics of flow itself for learning, LLM can also analyze the relationship between flow tokens, so it has better performance under the condition of less attack flow data.

3) *Comparison of Performance With Five Models on an Imbalanced Attack Dataset With Hybrid Attacks:* In order to better respond to the different situations of the main DDoS attacks in different clusters, the proportion of flow is modified again. Set the proportion of benign flow in each cluster to 80%, the

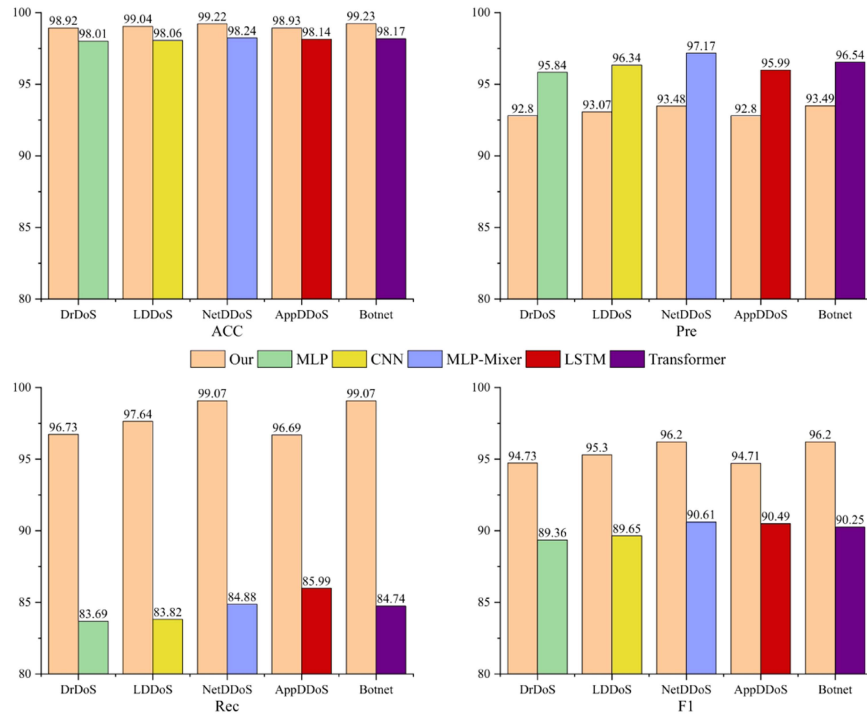


Fig. 6. Performance comparison on imbalanced attack data: Proposed scheme vs. five models.

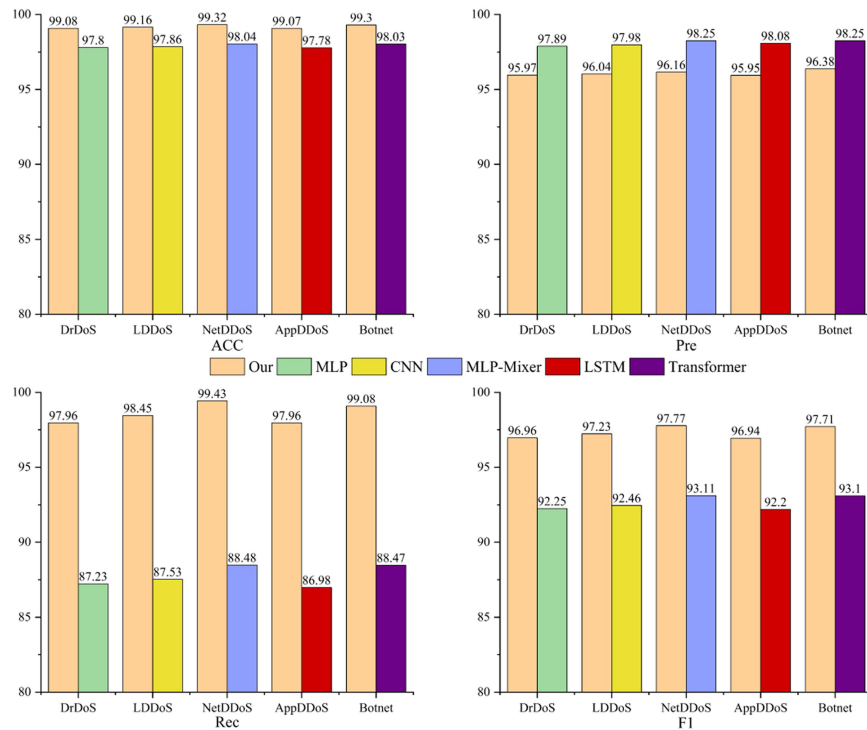


Fig. 7. Performance comparison on imbalanced attack data with hybrid attacks: Proposed scheme vs. five models.

proportion of main attack flow to 15%, and the proportion of the remaining four types of attack flow to 5%. The performance test of unbalanced and hybrid DDoS attacks is performed, and the results are shown in Fig. 7. It can be seen that the mixing of other non-major DDoS attacks has little effect on the detection performance of the proposed scheme, and due to the increase in the

proportion of attack flow, its detection performance is improved compared with the unbalanced data test results of 10% attacks. Compared with other detection schemes, it still maintains advantages in accuracy, recall and F1-score. The average recall rate is 12.35% higher, and the average F1-score is 5.07% higher. In addition, due to the addition of hybrid attacks, under the premise

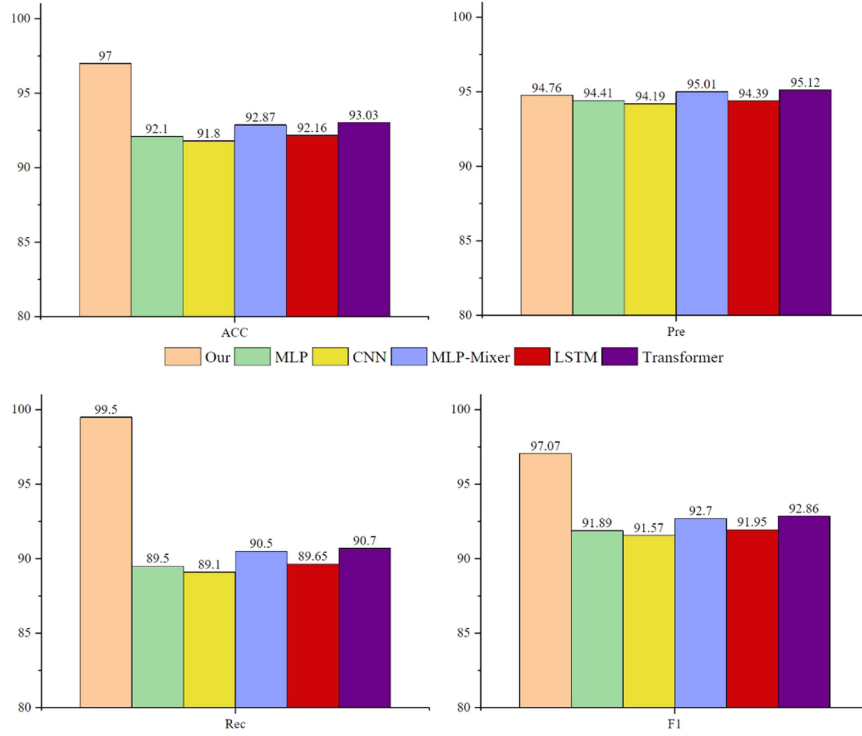


Fig. 8. Detection performance comparison on unknown datasets.

of decreasing imbalance, the accuracy of other schemes is still reduced, with an average decrease of 0.222%. The accuracy of our scheme has been improved, with an average increase of 0.118%. Although the increase is not significant, it can still be shown that the proposed scheme can adapt to scenarios with hybrid DDoS attacks.

The reason can be explained from two aspects: the number of attack features that can be used for learning and the ability to learn attack features. In the two cases of unbalanced and hybrid attacks, the number of attack flow provided to the model for learning attack features is small. The model does not have enough learning samples to learn the characteristics of attack flow, so it tends to judge attack flow as benign flow. The proposed scheme is based on the ability of flow order assignment and LLM to focus on the correlation between flows, which can better understand the differences and characteristics between step flows. In contrast, our scheme can pay attention to more features under the same condition of less attack flow. Therefore, it has better performance in unbalanced and hybrid attack scenarios.

4) *Comparison of Performance With Five Models on an Unknow DDoS Dataset:* In addition to being able to adapt to unbalanced and hybrid DDoS attack scenarios, we also hope that the proposed model can have good generalization ability to face unknown attacks. A total of 20,000 attacks of CI-CDDoS2019 and unknow attack types LDAP, SNMP, SSDP, ACK, Chargin and SNMP from the data set are selected, and 20,000 benign flow data are mixed to form a new data set that is unknown to the model. The data set is used for testing to reflect the generalization ability of the model. The training is based on the scene and data in Table III and the unknown data set is used for testing. The experimental results are presented in Fig. 8. Compared with conventional detection models,

the proposed scheme demonstrates significant improvements in accuracy, recall, and F1-score, achieving increases of 4.99%, 10.69%, and 5.29%, respectively. Furthermore, in comparison with the known and balanced attack tests, the accuracy, precision, recall and F1-score of other conventional models decreased by 5.83%, 3.28%, 8.84% and 5.83%, respectively. In contrast, the proposed model exhibited a more stable performance, with reductions of only 2.35%, 4.80%, -0.24% and 2.28%, respectively.

In the above experiments, the accuracy of the proposed model is lower than that of the other three performance. The reason for this is that the model can learn more knowledge to judge the flow as attack flow in a limited number of flow. In the face of less attack flow or unknown flow, the model tends to predict more positive samples, which means attack flow. Combined with the proposed model, it has a stronger ability to predict attack flow, so the accuracy is not high.

5) *Comparison of Performance With an LLM-Based Baseline:* In addition to being able to adapt to unbalanced and hybrid DDoS attack scenarios, we also hope that the proposed model can have good generalization ability to face unknown attacks. A total of 20,000 attacks of CI-CDDoS2019 and unknow attack types LDAP, SNMP, SSDP, ACK, Chargin and SNMP from the data set are selected, and 20,000 benign flow data are mixed to form a new data set that is unknown to the model. The data set is used for testing to reflect the generalization ability of the model. The training is based on the scene and data in Table III and the unknown data set is used for testing. The experimental results are presented in Fig. 8. Compared with conventional detection models,

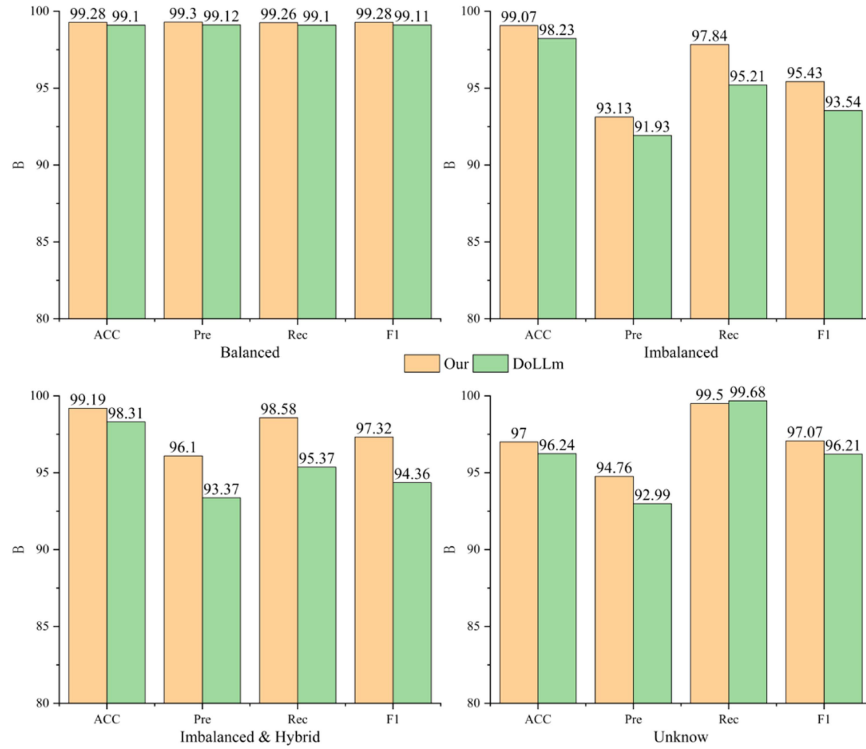


Fig. 9. Detection performance comparison with DoLLM on four kinds of datasets.

LLM extract richer information for DDoS detection. Moreover, the bottom-up architecture enables more targeted detection for different attack types. Therefore, the overall performance of the proposed scheme is superior to that of DoLLM.

C. Ablation Experiment

1) *Impact of Flow Order on Detection Performance:* In order to prove the role of flow order generation module, delete the flow order generation module, not in the form of a flow order matrix input by column. Use the flow data sorted in chronological order and slide the window with a length of 64 to generate a flow order and input it into the TGG. The balanced data set is used as the test data and the comparison results are shown in Fig. 10. After deleting the flow order generation module, the detection ability decreased significantly, and the accuracy rate, precision rate, recall rate and F1-score decreased by 3.99%, 2.04%, 5.79% and 3.86%, respectively. Because LLM's ability to focus on context is not utilized after deleting the flow order generation module, the correlation of DDoS flow data in the time dimension is not high. The order assignment module is designed according to the characteristics of the attack, which can make better use of the attention ability of the LLM context to learn, so as to obtain better detection performance.

2) *Impact of Masking Mechanism on Detection Performance:* The flow order matrix is used to input the flow data into the LLM by column input, which ensures that the input length is the same each time and does not need to be filled with padding token. The mask mechanism of LLM can be ignored so that it can pay attention to all the input flow data. As a comparison, we keep the order of the flow unchanged and randomly intercept the length of the input flow order. The length of interception is set to 4,8,16,32,48 and 64 respectively and it is filled to

ensure the consistency of the length and the mask mechanism is reintroduced. The balanced data set is used as the test data set and the comparison results are shown in Fig. 11. The results show that the accuracy rate, precision rate, recall rate and F1-score decrease by 9.19%, 9.35%, 9.20% and 9.21% respectively after using the mask mechanism. Because the mask mechanism prevents the model from learning all the flow data, the scope of its learning becomes smaller and the basis for judging attack flow is reduced. In the above experiments, the phenomenon that the normal flow is judged as the attack flow leads to the low accuracy rate also disappears. The existence of the mask can also hinder the continuity between flow data, frequent access to the same address or request the same protocol can be interrupted due to the presence of the mask. It is difficult for the model to learn enough knowledge from the relationship between flows to detect flow.

3) *Impact of TGG on Convergence Speed:* In order to verify the role of EAL-BT in generating TGG from the bottom up, it is proved whether the ability of the edge model is utilized in the cloud. Deleting the TGG in CE4LDD make it impossible to use the edge type to transform the dimension before the high-dimensional mapping of the flow order matrix. The flow order matrix is directly generated by the column token input to the embedding module. Comparing the training convergence speed of the embedding module before and after deleting the TGG, the training loss and verification loss during CE4LDD training are shown in Fig. 12. It can be seen that the convergence rate decreases after removing the TGG generated from the bottom-up EAL-BT process. This is because TGG contains the knowledge of edge model to deal with specific DDoS attacks. Before high-dimensional mapping, the flow can be vectorized according to the knowledge of edge model, so as to help CE4LDD training converge faster. After deleting

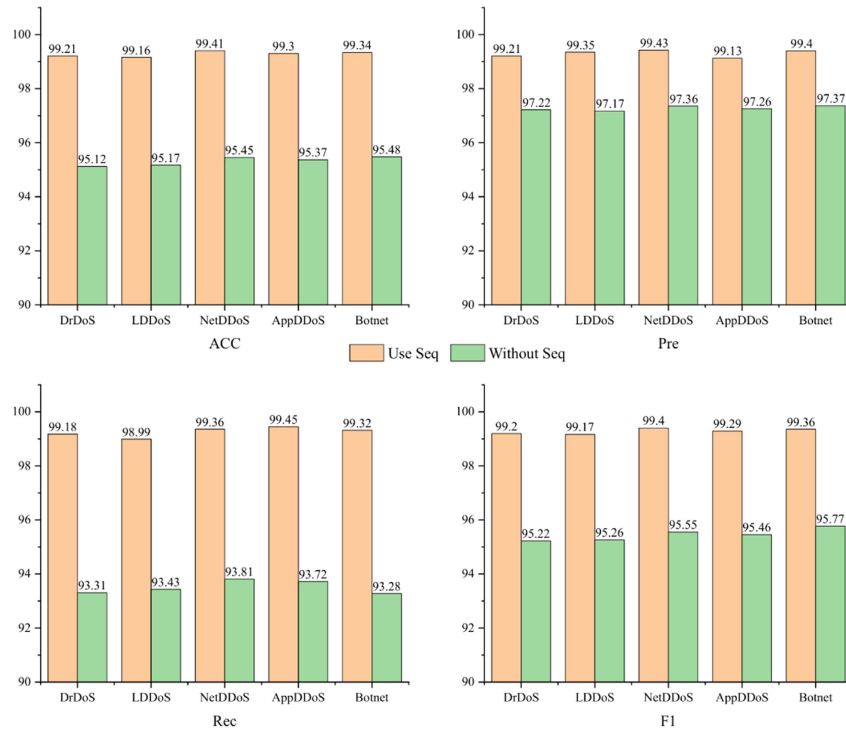


Fig. 10. Detection performance comparison without the flow order generation module.

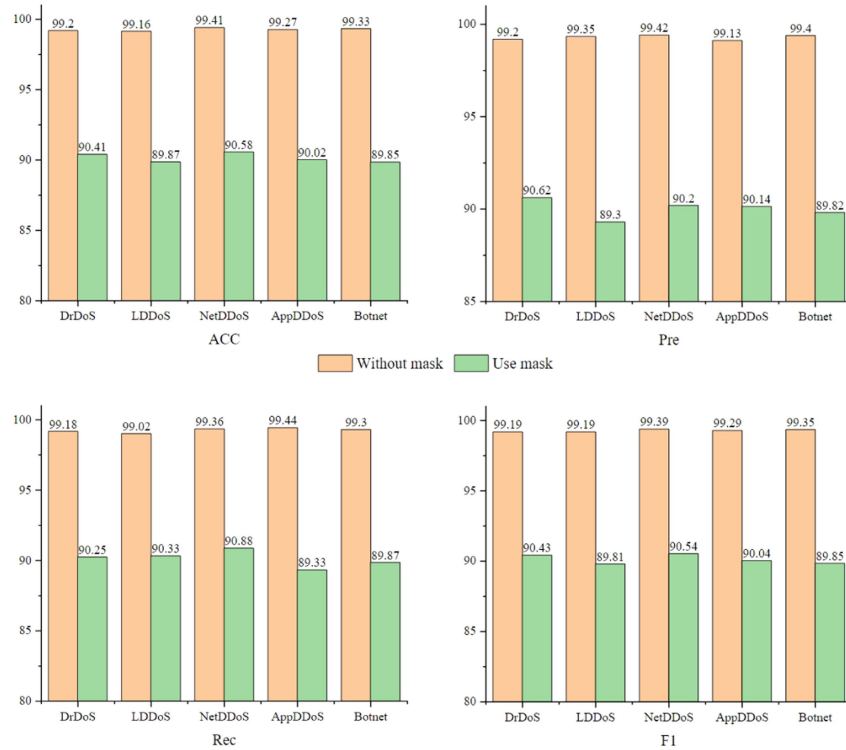


Fig. 11. Detection performance comparison with the mask mechanism.

TGG, this part of knowledge is wasted, which makes the initial loss value of CE4LDD higher and has a slower convergence speed. Moreover, using TGG reduces training time by 7.71% compared with not using it. This improvement is attributable to

TGG being pre-trained at the edge before cloud training; the knowledge acquired at the edge accelerates subsequent cloud training. Thus, cloud-edge collaboration reduces training overhead.

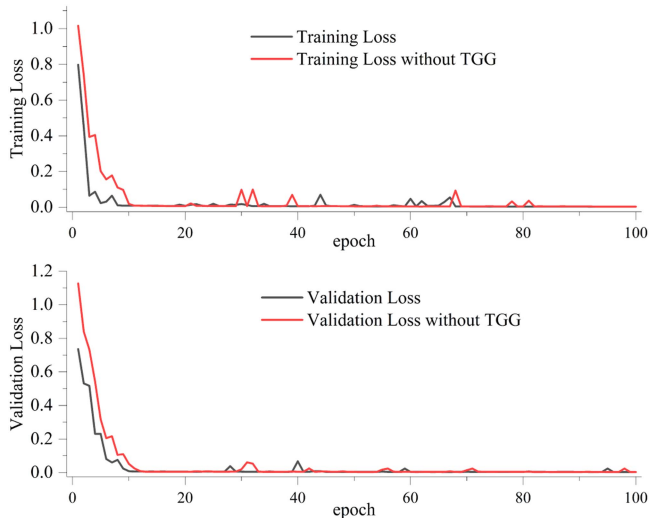


Fig. 12. Convergence speed comparison after removing TGG.

VI. CONCLUSION

This paper presents a multi-scenario DDoS detection for AIoT based on LLM. Firstly, a cloud-edge collaborative method called CE4LDD is proposed to realize LLM to detect DDoS attacks. The ability of LLM is utilized by assigning order to flow. TGG and embedding module are designed to enable LLM to process non-text type flow data. We formulate DDoS detection as a token-classification task within the LLM framework; final decisions are produced by a linear projection layer. Second, we propose EAL-BT, a bottom-up training and detection process that enables LLMs to adapt to multi-scenario DDoS attacks in AIoT and leverages edge-model capabilities to assist cloud-based LLM detection. Public data sets and self-data sets are used to simulate five scenarios of IoT applications that are subject to different DDoS attacks. Comparative experiments are carried out with six models for four different situations: balanced data, imbalanced data, hybrid attack data and unknown attack data. The results demonstrate that the proposed scheme has obvious advantages in detection performance, particularly in the latter three cases. Through ablation experiments, it is proved that the designed flow order generation module can improve the detection performance of DDoS. The designed TGG can speed up the training of the model. For future work, we will focus on optimizing transmission latency to improve cloud-edge adaptability, account for realistic network-traffic characteristics, and investigate methods for enabling LLM-based detection of encrypted traffic.

REFERENCES

- [1] K. Banti, M. Louta, and P. Baziana, "Data quality in human-centric sensing-based next-generation IoT systems: A comprehensive survey of models, issues, and challenges," *IEEE Open J. Commun. Soc.*, vol. 4, pp. 2286–2317, 2023.
- [2] R. Zhang et al., "Toward democratized generative AI in next-generation mobile edge networks," *IEEE Netw.*, vol. 39, no. 6, pp. 251–260, Nov. 2025, doi: [10.1109/MNET.2025.3541078](https://doi.org/10.1109/MNET.2025.3541078).
- [3] R. Zhang et al., "Interactive AI with retrieval-augmented generation for next generation networking," *IEEE Netw.*, vol. 38, no. 6, pp. 414–424, Nov. 2024.
- [4] R. Zhang, K. Xiong, Y. Lu, D. W. K. Ng, P. Fan, and K. B. Letaief, "SWIPT-Enabled cell-free massive MIMO-NOMA networks: A machine learning-based approach," *IEEE Trans. Wireless Commun.*, vol. 23, no. 7, pp. 6701–6718, Jul. 2024.
- [5] D. Wang et al., "An empirical study on the insecurity of end-of-life (EoL) IoT devices," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 4, pp. 3501–3514, Jul./Aug. 2024.
- [6] "Nexusguard Distributed denial of service (DDoS) trend report 2024," Nov. 2024. [Online] Available: <https://www.nexusguard.com/threat-report/ddos-trend-report-2024>
- [7] "Nokia threat intelligence report finds cybercriminal attacks on telco infrastructure are accelerating, driven by Generative AI and automation," Oct. 2024. [Online] Available: <https://www.nokia.com/about-us/news/releases/2024/10/02/nokia-threat-intelligence-report-finds-cybercriminal-attacks-on-telco-infrastructure-are-accelerating-driven-by-generative-ai-and-automation>
- [8] H. Wang, X. Di, Y. Wang, B. Ren, G. Gao, and J. Deng, "An intelligent digital twin method based on spatio-temporal feature fusion for IoT attack behavior identification," *IEEE J. Sel. Areas Commun.*, vol. 41, no. 11, pp. 3561–3572, Nov. 2023.
- [9] Y. Feng, W. Zhang, S. Yin, H. Tang, Y. Xiang, and Y. Zhang, "A collaborative stealthy DDoS detection method based on reinforcement learning at the edge of Internet of Things," *IEEE Internet Things J.*, vol. 10, no. 20, pp. 17934–17948, Oct. 2023.
- [10] Y. Liu, K. -F. Tsang, C. K. Wu, Y. Wei, H. Wang, and H. Zhu, "IEEE P2668-Compliant multi-layer IoT-DDoS defense system using deep reinforcement learning," *IEEE Trans. Consum. Electron.*, vol. 69, no. 1, pp. 49–64, Feb. 2023.
- [11] M. Lefoane, I. Ghafir, S. Kabir, I.-U. Awan, K. El Hindi, and A. Mahendran, "Latent semantic analysis and graph theory for alert correlation: A proposed approach for IoT botnet detection," *IEEE Open J. Commun. Soc.*, vol. 5, pp. 3904–3919, 2024.
- [12] K. -Y. Lam, S. Mitra, F. Gondesen, and X. Yi, "ANT-centric IoT security reference architecture—Security-by-design for satellite-enabled smart cities," *IEEE Internet Things J.*, vol. 9, no. 8, pp. 5895–5908, Apr. 2022.
- [13] Z. Li, Y. Kong, and C. Jiang, "A transfer double deep Q network based DDoS detection method for internet of vehicles," *IEEE Trans. Veh. Technol.*, vol. 72, no. 4, pp. 5317–5331, Apr. 2023.
- [14] M. Al-Fawa'reh, J. Abu-Khalaf, P. Szcwzyk, and J. J. Kang, "MalBoT-DRL: Malware botnet detection using deep reinforcement learning in IoT networks," *IEEE Internet Things J.*, vol. 11, no. 6, pp. 9610–9629, Mar. 2024.
- [15] A. Noroozian, E. T. Rodriguez, E. Lastdrager, T. Kasama, M. V. Eeten, and C. H. Gañán, "Can ISPs help mitigate IoT malware? A longitudinal study of broadband ISP security efforts," in *Proc. 2021 IEEE Eur. Symp. Secur. Privacy*, Vienna, Austria, 2021, pp. 337–352.
- [16] B. Ji et al., "Survey of secure communications of Internet of Things with artificial intelligence," *IEEE Internet Things Mag.*, vol. 5, no. 3, pp. 92–99, Sep. 2022.
- [17] H. Gu, L. Zhao, Z. Han, G. Zheng, and S. Song, "AI-Enhanced cloud-edge-terminal collaborative network: Survey, applications, and future directions," *IEEE Commun. Surveys Tuts.*, vol. 26, no. 2, pp. 1322–1385, Second Quarter 2024.
- [18] A. Hekmati, J. Zhang, T. Sarkar, N. Jethwa, E. Grippo, and B. Krishnamachari, "Correlation-aware neural networks for DDoS attack detection in IoT systems," *IEEE/ACM Trans. Netw.*, vol. 32, no. 5, pp. 3929–3944, Oct. 2024.
- [19] L. Zhao et al., "Shielding collaborative learning: Mitigating poisoning attacks through client-side detection," *IEEE Trans. Dependable Secure Comput.*, vol. 18, no. 5, pp. 2029–2041, Sep./Oct. 2021.
- [20] Y. Ma et al., "A survey of DDoS attack and defense technologies in multiaccess edge computing," *IEEE Internet Things J.*, vol. 12, no. 2, pp. 1428–1452, Jan. 2025.
- [21] M. Abbasi, A. Shahraki, J. Prieto, A. G. Arrieta, and J. M. Corchado, "Unleashing the potential of knowledge distillation for IoT flow classification," *IEEE Trans. Mach. Learn. Commun. Netw.*, vol. 2, pp. 221–239, 2024.
- [22] V. Pourahmadi, H. A. Alameddine, M. A. Salahuddin, and R. Boutaba, "Spotting anomalies at the edge: Outlier exposure-based cross-silo federated learning for DDoS detection," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 5, pp. 4002–4015, Sep./Oct. 2023.
- [23] F. Xing, F. Tong, J. Yang, G. Cheng, and S. He, "RAM: A resource-aware DDoS attack mitigation framework in clouds," *IEEE Trans. Cloud Comput.*, vol. 12, no. 4, pp. 1387–1400, Fourth Quarter 2024.
- [24] H. Zhu, P. Tiwari, A. Ghoneim, and M. S. Hossain, "A collaborative AI-Enabled pretrained language model for AIoT domain question answering," *IEEE Trans. Ind. Informat.*, vol. 18, no. 5, pp. 3387–3396, May 2022.
- [25] Y. -J. Liu et al., "A survey of integrating generative artificial intelligence and 6G mobile services: Architectures, solutions, technologies and outlooks," *IEEE Trans. Cogn. Commun. Netw.*, vol. 11, no. 3, pp. 1334–1356, Jun. 2025.

- [26] J. Xue, Y. Deng, Y. Gao, and Y. Li, "Auffusion: Leveraging the power of diffusion and large language models for text-to-audio generation," *IEEE/ACM Trans. Audio, Speech, Lang. Process.*, vol. 32, pp. 4700–4712, 2024.
- [27] Y. Li, B. Hu, X. Chen, L. Ma, Y. Xu, and M. Zhang, "LMEye: An interactive perception network for large language models," *IEEE Trans. Multimedia*, vol. 26, pp. 10952–10964, 2024.
- [28] B. Du, H. Du, D. Niyato, and R. Li, "Task-oriented semantic communication in large multimodal models-based vehicle networks," *IEEE Trans. Mobile Comput.*, vol. 24, no. 10, pp. 9822–9836, Oct. 2025, doi: [10.1109/TMC.2025.3564543](https://doi.org/10.1109/TMC.2025.3564543).
- [29] O. Friha, M. A. Ferrag, B. Kantarci, B. Cakmak, A. Ozgun, and N. Ghoualmi-Zine, "LLM-based edge intelligence: A comprehensive survey on architectures, applications, security and trustworthiness," *IEEE Open J. Commun. Soc.*, vol. 5, pp. 5799–5856, 2024.
- [30] R. Zhang et al., "Generative AI for space-air-ground integrated networks," *IEEE Wireless Commun.*, vol. 31, no. 6, pp. 10–20, Dec. 2024.
- [31] D. Rivkin et al., "AIoT smart home via autonomous LLM agents," *IEEE Internet Things J.*, vol. 12, no. 3, pp. 2458–2472, Feb. 2025.
- [32] R. Zhang et al., "Embodied AI-Enhanced vehicular networks: An integrated vision language models and reinforcement learning method," *IEEE Trans. Mobile Comput.*, vol. 24, no. 11, pp. 11494–11510, Nov. 2025, doi: [10.1109/TMC.2025.3582864](https://doi.org/10.1109/TMC.2025.3582864).
- [33] Y. Wang et al., "Large model based agents: State-of-the-art, cooperation paradigms, security and privacy, and future trends," *IEEE Commun. Surveys Tuts.*, early access, Jun. 3, 2025, doi: [10.1109/COMST.2025.3576176](https://doi.org/10.1109/COMST.2025.3576176).
- [34] H. Du et al., "Reinforcement learning with LLMs interaction for distributed diffusion model services," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 47, no. 10, pp. 8838–8855, Oct. 2025, doi: [10.1109/TPAMI.2025.3584698](https://doi.org/10.1109/TPAMI.2025.3584698).
- [35] L. Yang et al., "FuzzCoder: Code large language model-based fuzz testing for industrial IoT programs," *IEEE Internet Things J.*, vol. 12, no. 18, pp. 36842–36851, Sep. 2025, doi: [10.1109/IJOT.2025.3577602](https://doi.org/10.1109/IJOT.2025.3577602).
- [36] J. Wen et al., "Generative AI for low-carbon artificial intelligence of things with large language models," *IEEE Internet Things Mag.*, vol. 8, no. 1, pp. 82–91, Jan. 2025.
- [37] L. Hu, Z. Liu, Z. Zhao, L. Hou, L. Nie, and J. Li, "A survey of knowledge enhanced pre-trained language models," *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 4, pp. 1413–1430, Apr. 2024.
- [38] W. Fang, C. Zhu, and W. Zhang, "Toward secure and lightweight data transmission for cloud-edge-terminal collaboration in artificial intelligence of things," *IEEE Internet Things J.*, vol. 11, no. 1, pp. 105–113, Jan. 2024.
- [39] Q. Li et al., "DoLLM: How large language models understanding network flow data to detect carpet bombing DDoS," 2024, *arXiv:2405.07638*.
- [40] J. Zuo, Z. Guo, T. An, Z. Xu, and Y. Lu, "A security resilience metric framework based on the evolution of attack and defense scenarios," *IEEE Internet Things J.*, vol. 10, no. 19, pp. 17007–17021, Oct. 2023.
- [41] S. Wang et al., "Dissecting operational cellular IoT service security: Attacks and defenses," *IEEE/ACM Trans. Netw.*, vol. 32, no. 2, pp. 1229–1244, Apr. 2024.
- [42] C. Benzaid, T. Taleb, A. Sami, and O. Hireche, "FortisEDoS: A deep transfer learning-empowered economical denial of sustainability detection framework for cloud-native network slicing," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 4, pp. 2818–2835, Jul./Aug. 2024.
- [43] X. Lin et al., "ET-BERT: A contextualized datagram representation with pre-training transformers for encrypted flow classification," in *Proc. ACM Web Conf.*, 2022, pp. 633–642.
- [44] Y. Tian et al., "An edge-cloud collaboration framework for generative AI service provision with synergetic big cloud model and small edge models," *IEEE Netw.*, vol. 38, no. 5, pp. 37–46, Sep. 2024.
- [45] W. Zhang, D. Yang, C. Zhang, Q. Ye, H. Zhang, and X. Shen, "(Com)2Net: A novel communication and computation integrated network architecture," *IEEE Netw.*, vol. 38, no. 2, pp. 35–44, Mar. 2024.
- [46] S. Lakshminarayana and P. S. Thilagam, "Next-generation DDoS attacks on IoT deployments: Targeting the advanced features of MQTT v5.0 protocol," *IEEE Internet Things J.*, vol. 12, no. 12, pp. 22090–22109, Jun. 2025.
- [47] W. Zhang, P. Liao, D. Yang, Q. Ye, S. Mao, and H. Zhang, "Towards deterministic satellite-terrestrial integrated networks via resource adaptation and differentiated scheduling," *IEEE Trans. Mobile Comput.*, vol. 24, no. 10, pp. 11092–11109, Oct. 2025, doi: [10.1109/TMC.2025.3574740](https://doi.org/10.1109/TMC.2025.3574740).
- [48] X. He et al., "DNS rebinding threat modeling and security analysis for local area network of maritime transportation systems," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 2, pp. 2643–2655, Feb. 2023.
- [49] Z. Liu, J. Yu, B. Yan, and G. Wang, "A deep 1-D CNN and bidirectional LSTM ensemble model with arbitration mechanism for LDDoS attack detection," *IEEE Trans. Emerg. Topics Comput. Intell.*, vol. 6, no. 6, pp. 1396–1410, Feb. 2023.
- [50] W. Yin, et al., "DIAMOND: A structured coevolution feature optimization method for LDDoS detection in SDN-IoT," *Wireless Commun. Mobile Comput.*, vol. 1, Dec. 2021, Art. no. 9530274.
- [51] M. N. Kurt, Y. Yilmaz, X. Wang, and P. J. Mosterman, "Online privacy-preserving data-driven network anomaly detection," *IEEE J. Sel. Areas Commun.*, vol. 40, no. 3, pp. 982–998, Mar. 2022.
- [52] Z. Tian, C. Luo, J. Qiu, X. Du, and M. Guizani, "A distributed deep learning system for web attack detection on edge devices," *IEEE Trans. Ind. Informat.*, vol. 16, no. 3, pp. 1963–1971, Mar. 2020.
- [53] Y. Jia, "Security enhancement of IoT-based edge computing," Ph.D. dissertation, The George Washington Univ., Washington, DC, USA, 2021, doi: [10.5555/AAI28864213](https://doi.org/10.5555/AAI28864213).
- [54] X. Yan, Y. Xu, S. Yao, and Y. Sun, "A domain embedding model for botnet detection based on smart blockchain," *IEEE Internet Things J.*, vol. 11, no. 5, pp. 8005–8018, Mar. 2024.
- [55] W. Ma, X. Wang, J. Dong, M. Hu, and Q. Zhou, "A lightweight method for botnet detection in Internet of Things environment," *IEEE Trans. Netw. Sci. Eng.*, vol. 12, no. 4, pp. 2458–2472, Jul./Aug. 2025.
- [56] A. Sharma, S. Rani, S. H. Shah, R. Sharma, F. Yu, and M. M. Hassan, "An efficient hybrid deep learning model for denial of service detection in cyber physical systems," *IEEE Trans. Netw. Sci. Eng.*, vol. 10, no. 5, pp. 2419–2428, Sep./Oct. 2023.
- [57] A. Ali et al., "A novel homomorphic encryption and consortium blockchain-based hybrid deep learning model for industrial internet of medical things," *IEEE Trans. Netw. Sci. Eng.*, vol. 10, no. 5, pp. 2402–2418, Sep./Oct. 2023.
- [58] M. F. Saiyedand and I. Al-Anbagi, "Deep ensemble learning with pruning for DDoS attack detection in IoT networks," *IEEE Trans. Mach. Learn. Commun. Netw.*, vol. 2, pp. 596–616, 2024.
- [59] W. Zhang, Y. He, T. Zhang, C. Ying, and J. Kang, "Intelligent resource adaptation for diversified service requirements in industrial IoT," *IEEE Trans. Cogn. Commun. Netw.*, vol. 11, no. 4, pp. 2648–2661, Aug. 2025, doi: [10.1109/TCCN.2024.3502493](https://doi.org/10.1109/TCCN.2024.3502493).
- [60] H. Feng, W. Zhang, Y. Liu, Y. Li, J. Yin, and J. Kang, "GNN-enabled multi-agent DRL for adaptive path selection in multi-network domains," *IEEE Trans. Netw. Sci. Eng.*, vol. 13, pp. 130–145, Jun. 2025, doi: [10.1109/TNSE.2025.3582258](https://doi.org/10.1109/TNSE.2025.3582258).
- [61] S. Shen, C. Cai, Y. Shen, X. Wu, W. Ke, and S. Yu, "MFGD3QN: Enhancing edge intelligence defense against DDoS with mean-field games and dueling double deep Q-network," *IEEE Internet Things J.*, vol. 11, no. 13, pp. 23931–23945, Jul. 2024.
- [62] Z. Zhao et al., "FOSS: Towards fine-grained unknown class detection against the open-set attack spectrum with variable legitimate flow," *IEEE/ACM Trans. Netw.*, vol. 32, no. 5, pp. 3945–3960, Oct. 2024.
- [63] J. Li, L. Lyu, X. Liu, X. Zhang, and X. Lyu, "FLEAM: A federated learning empowered architecture to mitigate DDoS in industrial IoT," *IEEE Trans. Ind. Informat.*, vol. 18, no. 6, pp. 4059–4068, Jun. 2022.
- [64] R. F. Hayat, S. Aurangzeb, M. Aleem, G. Srivastava, and J. C. -W. Lin, "ML-DDoS: A blockchain-based multilevel DDoS mitigation mechanism for IoT environments," *IEEE Trans. Eng. Manag.*, vol. 71, pp. 12605–12618, 2024.
- [65] G. Sun et al., "Large language model (LLM)-Enabled graphs in dynamic networking," *IEEE Netw.*, vol. 39, no. 4, pp. 290–301, Jul. 2025.
- [66] W. Zhang, N. Tang, D. Yang, R. Guo, H. Zhang, and X. Shen, "Det(Com)2: Deterministic communication and computation integration toward AIGC services," *IEEE Wireless Commun.*, vol. 31, no. 3, pp. 32–41, Jun. 2024.
- [67] I. Lotfi, N. Alabbasi, and O. Alhussein, "Rethinking strategic mechanism design in the age of large language models: New directions for communication systems," *IEEE Internet Things Mag.*, vol. 8, no. 6, pp. 50–57, 5, Nov. 2025, doi: [10.1109/MIOT.2025.3576260](https://doi.org/10.1109/MIOT.2025.3576260).
- [68] R. Zhang, K. Xiong, Y. Lu, B. Gao, P. Fan, and K. B. Letaief, "Joint coordinated beamforming and power splitting ratio optimization in MU-MISO SWIPT-Enabled HetNets: A multi-agent DDQN-Based approach," *IEEE J. Sel. Areas Commun.*, vol. 40, no. 2, pp. 677–693, Feb. 2022.
- [69] J. Liu, T. Li, Q. Wang, Y. Wang, Z. Guo, and K. Yu, "Unleashing collaborative potentials: Multifaceted collaboration among agents in multitask Internet of Things networks," *IEEE Internet Things J.*, vol. 12, no. 14, pp. 28121–28135, Jul. 2025.

- [70] H. Cui, Y. Du, Q. Yang, Y. Shao, and S. C. Liew, "LLMind: Orchestrating AI and IoT with LLM for complex task execution," *IEEE Commun. Mag.*, vol. 63, no. 4, pp. 214–220, Apr. 2025, doi: [10.1109/MCOM.002.2400106](https://doi.org/10.1109/MCOM.002.2400106).
- [71] N. Zhong et al., "CASIT: Collective intelligent agent system for Internet of Things," *IEEE Internet Things J.*, vol. 11, no. 11, pp. 19646–19656, Jun. 2024.
- [72] B. Xiao, B. Kantarci, J. Kang, D. Niyato, and M. Guizani, "Efficient prompting for LLM-based generative Internet of Things," *IEEE Internet Things J.*, vol. 12, no. 1, pp. 778–791, Jan. 2025.
- [73] T. Tu et al., "Towards lifelong unseen task processing with a lightweight unlabeled data schema for AIoT," *IEEE Internet Things J.*, vol. 12, no. 4, pp. 3441–3452, Feb. 2025.
- [74] N.-B. V. Le and J.-H. Huh, "AgTech: Building smart aquaculture assistant system integrated IoT and Big Data analysis," *IEEE Trans. AgriFood Electron.*, vol. 2, no. 2, pp. 471–482, Sep./Oct. 2024.
- [75] A. Selim, J. Zhao, and B. Yang, "Large language model for smart inverter cyber-attack detection via textual analysis of Volt/VAR commands," *IEEE Trans. Smart Grid*, vol. 15, no. 6, pp. 6179–6182, Nov. 2024.
- [76] B. Karunanayake, I. Khalil, X. Yi, and K. -Y. Lam, "Toward LLM-driven adaptive policy orchestration for host-based intrusion detection systems in IoT environments," *IEEE Netw.*, vol. 39, no. 5, pp. 66–73, Sep. 2025, doi: [10.1109/MNET.2025.3579532](https://doi.org/10.1109/MNET.2025.3579532).
- [77] X. Chen, W. Wu, L. Li, and F. Ji, "LLM-Empowered IoT for 6G networks: Architecture, challenges, and solutions," *IEEE Internet Things Mag.*, vol. 8, no. 6, pp. 34–41, Nov. 2025, doi: [10.1109/MIOT.2025.3582641](https://doi.org/10.1109/MIOT.2025.3582641).
- [78] M. Fu, P. Wang, M. Liu, Z. Zhang, and X. Zhou, "IoV-BERT-IDS: Hybrid network intrusion detection system in IoV using large language models," *IEEE Trans. Veh. Technol.*, vol. 74, no. 2, pp. 1909–1921, Feb. 2025.
- [79] Q. Liu, J. Mu, D. Chen, R. Zhang, Y. Liu, and T. Hong, "LLM enhanced reconfigurable intelligent surface for energy-efficient and reliable 6G IoV," *IEEE Trans. Veh. Technol.*, vol. 74, no. 2, pp. 1830–1838, Feb. 2025.
- [80] R. Zhang et al., "Generative AI-Enabled vehicular networks: Fundamentals, framework, and case study," *IEEE Netw.*, vol. 38, no. 4, pp. 259–267, Jul. 2024.
- [81] G. Sun et al., "Generative AI for advanced UAV networking," *IEEE Netw.*, vol. 39, no. 4, pp. 244–253, Jul. 2025.
- [82] J. Hu et al., "Federated large language model: Solutions, challenges and future directions," *IEEE Wireless Commun.*, vol. 32, no. 4, pp. 82–89, Aug. 2025, doi: [10.1109/MWC.009.2400244](https://doi.org/10.1109/MWC.009.2400244).
- [83] S. Han et al., "FedSecurity: A benchmark for attacks and defenses in federated learning and federated LLMs," in *Proc. 30th ACM SIGKDD Conf. Knowl. Discov. Data Mining*, New York, NY, USA, pp. 5070–5081, 2024.
- [84] F. Wu et al., "FedBiOT: A solution for federated large language model fine-tuning with intellectual property protection," 2023. [Online] Available: <https://openreview.net/forum?id=i5da6iedW8>
- [85] C. Han, T. Yang, Z. Cui, and X. Sun, "A privacy-preserving and trustworthy inference framework for LLM-IoT integration via hierarchical federated collaborative computing," *IEEE Internet Things J.*, vol. 12, no. 24, pp. 51877–51891, Dec. 2025.
- [86] W. Zhang, T. Sun, D. Yang, T. H. Luan, and H. Zhang, "Dynamic resource scheduling for deterministic communication, computation, and control integration in industrial cyber-physical systems," *IEEE Trans. Cogn. Commun. Netw.*, early access, May 15, 2025, doi: [10.1109/TCCN.2025.3570438](https://doi.org/10.1109/TCCN.2025.3570438).
- [87] Y. Liu et al., "Brain-like cognition-driven model factory for IIoT fault diagnosis by combining LLMs with small models," *IEEE Internet Things J.*, vol. 12, no. 16, pp. 32296–32309, Aug. 2025, doi: [10.1109/IIOT.2024.3503274](https://doi.org/10.1109/IIOT.2024.3503274).
- [88] Y. Xia et al., "FCLLM-DT: Empowering federated continual learning with large language models for digital-twin-based industrial IoT," *IEEE Internet Things J.*, vol. 12, no. 6, pp. 6070–6081, Mar. 15, 2025.
- [89] A. Praseed and P. S. Thilagam, "Modelling behavioural dynamics for asymmetric application layer DDoS detection," *IEEE Trans. Inf. Forensics Secur.*, vol. 16, pp. 617–626, 2021.
- [90] S. Vaishali et al., "SYNTROPY: TCP SYN DDoS attack detection for software defined network based on Rényi entropy," *Comput. Netw.*, 244, Art. no. 110327, 2024.
- [91] D. Tang, Y. Yan, S. Zhang, J. Chen, and Z. Qin, "Performance and features: Mitigating the low-rate TCP-Targeted DoS attack via SDN," *IEEE J. Sel. Areas Commun.*, vol. 40, no. 1, pp. 428–444, Jan. 2022.
- [92] A. M. Eldhai et al., "Improved feature selection and stream traffic classification based on machine learning in software-defined networks," *IEEE Access*, vol. 12, pp. 34141–34159, 2024.
- [93] R. Zhang et al., "Generative AI agents with large language model for satellite networks via a mixture of experts transmission," *IEEE J. Sel. Areas Commun.*, vol. 42, no. 12, pp. 3581–3596, Dec. 2024.
- [94] I. Sharafaldin et al., "Developing realistic distributed denial of service (DDoS) attack dataset and taxonomy," in *Proc. Int. Carnahan Conf. Secur. Technol.*, 2019, pp. 1–8.
- [95] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. 4th Int. Conf. Inf. Syst. Secur. Privac.*, 2018, pp. 108–116.
- [96] O. Liu et al., "Multi-domain malicious behavior knowledge base framework for multi-type DDoS behavior detection," *Intell. Automat. Soft Comput.*, vol. 37, no. 3, pp. 2955–2977, 2023.
- [97] H. Touvron et al., "LLaMA: Open and efficient foundation language models," 2023, *arXiv:2302.13971*.



Aoran Huang (Graduate Student Member, IEEE) was born in Liaoning, China. He received the B.S. degree from the North China University of Technology, Beijing, China, in 2021. He is currently working toward the Ph.D. degree in information and telecommunications engineering. He joined the National Engineering Research Center for Advanced Network Technologies, Beijing Jiaotong University, Beijing. His research interests include the architecture of next-generation internet, network security, intelligent communication networks, and large language model.



Jingfu Yan received the B.S. degree from Anhui University, Hefei, China, in 2018, and the M.S. degree from Beijing Jiaotong University, Beijing, China, in 2022, where he is currently working toward the Ph.D. degree with the National Engineering Research Center for Advanced Network Technologies. His research interests include network security, next generation internet, and intelligent networks.



Xiaojing Fan received the M.S. degree in electronic information from Beijing Jiaotong University (BJTU), China, in 2023. She is currently working toward the Ph.D. degree in information and communication engineering. She joined the National Engineering Research Center for Advanced Network Technologies, BJTU. Her research interest includes the architecture of next-generation internet, satellite communications, and artificial intelligence.



Huachun Zhou received the B.S. degree from the People's Police Officer University of China, in 1986, and the M.S. degree in telecommunication automation and the Ph.D. degree in telecommunications and information systems from Beijing Jiaotong University (BJTU), Beijing, China, in 1989 and 2008, respectively. He is currently a Professor with the National Engineering Research Center for Advanced Network Technologies, BJTU. He has authored more than 40 peer-reviewed papers and is also the holder of 17 patents. His research interests mainly include the areas of Internet architecture, space-air-ground integrated networks, intelligent communication networks, mobility management, mobile and secure computing, and routing protocols.